Implementação de uma plataforma de transmissão ao vivo

OnlyHighIQ

2025/2025

1220788 - Pedro Miguel Ferreira Laranjeira da Rocha

Implementação de uma plataforma de transmissão ao vivo

OnlyHighIQ

2025/2025

1220788 - Pedro Miguel Ferreira Laranjeira da Rocha

Licenciatura em Engenharia Informática

Setembro de 2025

Orientador ISEP: Paula Correia Tavares

Supervisor Externo: Carlos Lima

Declaração de Integridade

Declaro ter conduzido este trabalho académico com integridade.

Não plagiei ou apliquei qualquer forma de uso indevido de informações ou falsificação de resultados ao longo do processo que levou à sua elaboração.

Portanto, o trabalho apresentado neste documento é original e de minha autoria, não tendo sido utilizado anteriormente para nenhum outro fim.

Declaro ainda que tenho pleno conhecimento do Código de Conduta Ética do P.PORTO.

ISEP, 2025/2025

Dedicatória

Dedico este trabalho à minha avó

Resumo

Este relatório documenta o desenvolvimento de uma plataforma de transmissão em direto no âmbito da unidade curricular Projeto/Estágio (PESTI) do curso de Engenharia Informática do Instituto Superior de Engenharia do Porto, concebida no escritorio da empresa OnlyHighIQ. O projeto nasce da ambição de simplificar a experiência de *livestreaming*, reduzindo a dependência de ferramentas de terceiros e tornando o processo acessível a criadores com níveis de literacia tecnológica distintos.

A solução materializou-se na criação de canais dedicados por criador, com controlo de acesso por *tiers*, e na integração de mecanismos de interação com a comunidade. Foram desenvolvidas interfaces específicas para criadores e para utilizadores, um *player* responsivo com um chat de baixa latência suportado por WebSockets. A arquitetura, ancorada em boas práticas de engenharia de software e em metodologia *Scrum*, foi validada através de testes de domínio e de ponta a ponta (end-to-end).

Os resultados atingidos confirmam os objetivos delineados, e reforçam competências técnicas e profissionais que culminaram na entrega de um *MVP* funcional. Embora persistam algumas limitações e um conjunto claro de melhorias a realizar, o projeto demonstrou a viabilidade de incorporar funcionalidades de *livestreaming* numa plataforma própria.

Palavras-chave (Tema): Transmissão ao vivo, Interação em tempo real, Rede social.

Palavras-chave (Tecnologias): AWS, RTMP, React, CI/CD.

Abstract

This report documents the development of a live-streaming platform within the scope of the Projeto/Estágio (PESTI) curricular unit of the Bachelor's Degree in Informatics Engineering at the Instituto Superior de Engenharia do Porto, conceived at the office of the company OnlyHighIQ. The project stems from the ambition to simplify the *livestreaming* experience, reducing dependence on third-party tools and making the process accessible to creators with different levels of technological literacy.

The solution materialised in the creation of dedicated channels per creator, with access control by *tiers*, and in the integration of mechanisms for interaction with the community. Specific interfaces for creators and for users were developed, as well as a responsive *player* with a low-latency chat supported by WebSockets. The architecture, anchored in software engineering best practices and in the *Scrum* methodology, was validated through domain tests and end-to-end testing.

The results achieved confirm the outlined objectives and strengthen technical and professional competences that culminated in the delivery of a functional *MVP*. Although some limitations persist and there is a clear set of improvements to be made, the project demonstrated the feasibility of incorporating *livestreaming* functionalities into a proprietary platform.

Keywords (Theme): Livestreaming, Real Time Interaction, Social Media.

Keywords (Tecnologies): AWS, RTMP, React, CI/CD.

Agradecimentos

Começo por agradecer à professora Paula Tavares, pela sua disponibilidade para acompanhar o meu trabalho e por ajudar sempre que necessário.

Agradeço à OnlyHighIQ pela oportunidade e pela confiança que depositou nos meus ombros, sinto que dentro da empresa cresci, convivi, ri e conheci pessoas que espero levar para o resto da minha jornada. Devo um especial agradecimento ao meu supervisor externo, Carlos Lima, que conseguiu retirar o melhor de mim, colocando sempre o meu bem estar em primeiro lugar.

Quero agradecer ao Instituto Superior de Engenharia do Porto pelas oportunidades dadas nestes longos três anos, pela exelente qualidade de formação, e pela escolha brilhande de funcionarios e docentes.

Num plano mais íntimo, agradeço à minha família pelo amor e confiança que me deram.

Ao meu pai, pela sua sabedoria, pela força e pela perseverança. Obrigado por sempre me ensinares a importância de lutar pelos meus sonhos. Sempre me disseste para nunca deixar de estudar, porque querias um futuro melhor para mim, mas nunca soube como responder a esse comentario vindo de um homem que nunca me deixou sem apoio.

À minha mãe, pelo carinho, paciência e amor que me mostrou. A tua dedicação e esforço foram fundamentais para eu conhecer as verdadeiras cores do mundo. Sou grato por tudo o que fizeste por mim, mesmo nos momentos mais difíceis.

Aos meus amigos Tiago Marques, Bruno Lourenço, Vasco Martins, Pedro Fonseca e Gaspar Lima, poucas palavras descrevem o carinho e a alegria que sinto por vocês, apenas quero que saibam que não seria o mesmo Pedro, alegre e sorridente que conhecem sem vocês.

Conteúdo

Agradecimentos **ix**

Lista de Figuras **xv**

Lista de Tabelas **xvii**

Lista de Código **xix**

Lista de Acrónimos **xxi**

1 Introdução **1**
1.1 Enquadramento/Contexto 1
1.2 Descrição do Problema 2
1.2.1 Objetivos 2
1.2.2 Abordagem 3
1.2.3 Contributos 4
1.2.4 Planeamento do trabalho 4
1.3 Estrutura do relatório 5

2 Estado da Arte **7**
2.1 Trabalhos relacionados 7
2.1.1 Youtube 7
2.1.2 KICK 8
2.1.3 Twitch 10
2.1.4 TikTok 11
2.1.5 Discord 12
2.1.6 Tabela Comparativa das Plataformas 13
2.2 Tecnologias existentes 14
2.2.1 Protocolos De *Streaming* 14
2.2.2 Back-End 14
2.2.3 Front-End 15
2.2.4 Base de Dados 15
2.3 Análise e seleção das tecnologias 16
2.3.1 Protocolos de Streaming 16
2.3.2 Back-end 17
2.3.3 Front-end 17
2.3.4 Base de dados 17
2.4 Conclusão 17

3 Análise e desenho da solução **19**
3.1 Domínio do problema 19

3.1.1 Glossário do Domínio 19
3.1.2 Modelo de Domínio 20
3.2 Requisitos Funcionais e não funcionais 21
3.2.1 Requisitos Funcionais 22
3.2.2 Requisitos Não Funcionais 22
FURPS+ 22
3.3 Casos de Uso 23
3.4 Desenho 24
3.4.1 Modelo C4 24
3.4.2 Nível 1 24
3.4.3 Nível 2 25
3.4.4 Nível 3 26
3.4.5 Vista Física 29
3.4.6 Modelo de Dados 31
3.4.7 Diagramas de sequência dos casos de uso 33
Casos de Uso **C1** & **C2** 33
Casos de Uso **C6** & **C7** & **C3** 34
Casos de Uso **C8** & **C9** 35
Casos de Uso **C5** & **C10** 36
Caso de Uso **C11** 37
Caso de Uso **C12** 38
Caso de Uso **C13** 39
Caso de Uso **C14** 40
Caso de Uso **V1** 41
Caso de Uso **V2** 42
Caso de Uso **V3** 43
3.4.8 Alternativas ao *design* 44
3.5 Conclusão 44

4 Implementação da Solução **45**
4.1 Descrição da implementação 45
4.1.1 **Low Latency Chat** 45
4.1.2 **Tiers** 48
4.1.3 **Livestream Menu** 49
4.1.4 **Livestream** 50
4.2 Testes 54
Testes de Domínio 54
Testes *end-2-end* 56
4.3 Avaliação da solução 57

5 Conclusões **59**
5.1 Objetivos concretizados 59
5.2 Limitações e trabalho futuro 59
5.3 Apreciação final 59

A Diagrama de casos de uso e Diagramas de sequencia **63**

Lista de Figuras

1.1 Pilares e Valores Da Metodologia SCRUM [2] 3

2.1 Interface de Criador de Conteúdo do Youtube Live **Youtube** 8
2.2 Interface de utilizador da Kick [5] 9
2.3 Interface de Criador de Conteúdo da Twitch [7] 10
2.4 *LIVE* center do **TikTok** 11
2.5 Escolha de cenário para transmissão 12

3.1 Modelo de Dominio - Streaming 21
3.2 Vista Lógica do Sistema - Nível 1 25
3.3 Vista Lógica do Sistema - Nível 2 26
3.4 Vista Lógica do *Back-End* - Nível 3 26
3.5 Vista de implementação 28
3.6 Vista Lógica Nível 3 - *Front-End* 29
3.7 Vista física - Infraestrutura AWS 30
3.8 Modelo de Dados 32
3.9 Diagrama de sequência do sistema para criar título e descrição da transmissão. 34
3.10 Diagrama de sequência do sistema para configurar câmara e microfone (ativação, permissões e teste). 35
3.11 Diagrama de sequência do sistema para desativar/reativar câmara e microfone durante a transmissão. 36
3.12 Diagrama de sequência do sistema para iniciar a transmissão (*start stream*). 36
3.13 Diagrama de sequência do sistema para terminar a transmissão (*stop stream*). 37
3.14 Diagrama de sequência do sistema para visualização do chat pelo criador. . 38
3.15 Diagrama de sequência do sistema para interação do criador no chat (envio e moderação). 39
3.16 Diagrama de sequência do sistema para gestão e apresentação do *Live Menu*. 40
3.17 Diagrama de sequência do sistema para gestão e seleção de *tiers* de acesso. 41
3.18 Diagrama de sequência do sistema para visualização da transmissão pelo utilizador. 42
3.19 Diagrama de sequência do sistema para interação do utilizador no chat. . . 43
3.20 Diagrama de sequência do sistema para controlos de reprodução da transmissão pelo utilizador. 43

4.1 Componente do *low latency chat* 48
4.2 Edição e *Preview* de um escalão de acesso 48
4.3 Área de seleção de um escalão 49
4.4 Menu de *Items* - Visualizador 49
4.5 Menu de *Items* - Criador 50
4.6 Interface de Criador 52
4.7 Interface de Utilizador 53
4.8 Testes de Domínio 55

4.9 Teste de Carga 58

A.1 Diagrama de casos de uso - Viewer 63
A.2 Diagrama de casos de uso - Creator Configuração 64
A.3 Diagrama de casos de uso - Creator Interação 65
A.4 Diagrama de Sequência completo 66
A.5 Diagrama de Sequência - *Front-End* 67
A.6 Diagrama de Sequência - *Back-End* 68

Lista de Tabelas

1.1 Missão, Visão e Valores da OnlyHighIQ 2
1.2 Divisão de tempo no projeto 5

2.1 Tabela Comparativa das Plataformas 13
2.2 Resumo comparativo de protocolos de *streaming* 14
2.3 Tabela resumo das tecnologias *back-end* (BE) 15
2.4 Tabela resumo das tecnologias *front-end* (FE) 15
2.5 SGBD e tecnologias de dados mais comuns 16
2.6 Comparação entre React e Angular 17

3.1 Glossário do domínio da aplicação 20
3.2 Requisitos Funcionais 22
3.3 Casos de uso do ator *Creator* A.2 & A.3 24
3.4 Casos de uso do ator *Viewer* A.1 24
3.5 Serviços disponibilizados para o consumo do sistema 25
3.6 Serviços AWS e respetivo papel na solução 31

5.1 Grau de Conclusão dos Objetivos 59

Lista de Código

4.1 Ligação ao Chat 45
4.2 Remoção do Chat 46
4.3 Envio das Mensagens 47
4.4 Criação do Canal - IVS 50
4.5 Classe de dominio - LiveStreamInfo 51
4.6 Retirar a *MediaStream* do navegador 52
4.7 Teste de criação do LiveStreamInfo 55
4.8 Criação do ambiente de testes 56
4.9 Configuração do ambiente de testes 56
4.10 Operações de inicio e fim de transmissão 57
4.11 Teste de inicio e fim de transmissão 57

Lista de Acrónimos

ACID	Atomicity, Consistency, Isolation, Durability.
ACM	AWS Certificate Manager.
ALB	Application Load Balancer.
API	Application Programming Interface.
AWS	Amazon Web Services.
CI/CD	Continuous Delivery/Continuous deployment.
CRUD	Create, Read, Update and Delete.
DIP	Dependency Inversion Principle.
DTO	Data Transfer Object.
FURPS+	Functionality, Usability, Reliability, Performance,Supportability.
HLS	HTTP Live Streaming.
I/O	Input/Output.
IAM	Identity and Access Management.
IVS	Interactive Video Service.
MVP	Minimum Viable Product.
PESTI	Projecto / Estágio.
RTMP	Real-Time Messaging Protocol.
RTP	Real-time Transport Protocol.
SDK	Software Development Kit.
SPA	Single Page Application.
SSD	System Sequence Diagram.
SSL	Secure Sockets Layer.
SSR	Server Side Rendering.
TLS	Transport Layer Security.
UC	Use Case.
UC	Unidade Curricular.
UDP	User Datagram Protocol.
UML	Unified Modeling Language.
US	User Story.

VPC	Virtual Private Cloud.
WAF	Web Application Firewall.

Capítulo 1

Introdução

Neste capítulo inicial, é fornecido ao leitor um contexto amplo, facilitando a interpretação dos assuntos descritos ao longo deste relatório.

A introdução organiza-se em torno do enquadramento e do contexto do projeto. Nele apresentam-se os aspetos iniciais essenciais: o problema em estudo, os objetivos e a abordagem adotada. Por fim, o capítulo explicita os contributos do projeto, o planeamento e a estrutura geral do relatório.

1.1 Enquadramento/Contexto

O presente projeto surge no âmbito da Unidade Curricular (UC) Projecto / Estágio (PESTI) da Licenciatura em Engenharia Informática do Instituto Superior de Engenharia do Porto e foi desenvolvido no escritório da *OnlyHighIQ*.

Este projeto parte de uma interpretação rigorosa do problema e aplica os conhecimentos adquiridos e as boas práticas de engenharia informática. Sintetiza o estado da arte e compara abordagens alternativas. Após esta avaliação, desenha-se e implementa-se a solução, sustentada por testes e documentação. Por fim, registam-se e apresentam-se com rigor os processos, decisões e resultados.

A *OnlyHighIQ* é uma empresa fundada recentemente, encontra-se no coração da cidade do Porto, a sua localização oferece um enquadramento dinâmico e inspirador à inovação e à colaboração. Inserida numa zona vibrante, reconhecida pela sua rica história e relevância cultural, a área garante acesso facilitado a comodidades essenciais e às principais ligações de transporte, constituindo um polo conveniente para clientes e parceiros. [1]

A missão, a visão e os valores são os pilares fundamentais para definir a identidade e o propósito de uma organização.

Tabela 1.1: Missão, Visão e Valores da OnlyHighIQ

Missão	"*...to create a dynamic community where brilliant minds collaborate to develop cutting-edge solutions and innovative platforms that transform visionary ideas into reality.*"[1]
Visão	"*To create a limitless innovation landscape where the most brilliant minds, have the tools to build the future—ensuring that progress and opportunity are distributed to everyone, everywhere.*"[1]
Valores	"*Innovation Without Borders, Collaborative Excellence, Impact with Purpose, Courage to Disrupt, Principle-Led Integrity*"[1]

A **OnlyYes** é o seu primeiro projeto que surge como ecossistema principal da empresa mãe, o seu foco é principalmente na criação de conteúdo pago.

Neste relatório vão ser focados aspetos da propagação de conteúdo de baixa latência, mas é importante realçar que a solução implementada passa por toda a infraestrutura. A solução apresentada neste relatório, foi apenas uma etapa entre vários processos e componentes, e decisões que levarão a **OnlyYes** ao lançamento.

O meu percurso na **OnlyHighIQ** decorreu durante cerca de quatro meses, período em que fui instruído de planear, conceber e discutir a arquitetura global da aplicação, de acordo com as boas práticas transmitidas na formação **AWS Cloud Architect** ministrada pela empresa, bem como com os padrões e conhecimentos que fui adquirindo ao longo do meu percurso.

1.2 Descrição do Problema

Atualmente, no mercado de criação de conteúdo encontram-se diversas barreiras técnicas para pessoas com dificuldades tecnológicas. Muitos dos criadores de conteúdo, dependem de ferramentas externas para realizar uma transmissão ao vivo, então a **OnlyYes** quer implementar uma rede social, onde seja possível realizar uma *livestream* com poucos cliques e pré-requisitos.

Apontou-se, também, a necessidade dos criadores de conteúdo conseguirem comunicar com os utilizadores da plataforma, pelo que foi necessário realizar um *low latency chat*.

O problema resume-se à criação de uma experiência unificada de *livestream* e comunicação em tempo real, acessível em poucos cliques, sem depender de ferramentas externas.

1.2.1 Objetivos

Para a solução a desenvolver, definiu-se os seguintes objetivos:

- **Modelação e criação de escalões de acesso para controlar o acesso à transmissão**: após a criação do perfil privado, são criados escalões com diferentes preços e benefícios.
- **Criação de canal único por criador**: provisionar um canal exclusivo a cada criador.
- **Desenvolvimento da interface do criador**: *dashboard* para iniciar e terminar a transmissão, escolher câmara e microfone, pré-visualizar o vídeo, definir título e descrição, editar um menu de doações, e visualizar métricas.

- **Desenvolvimento da interface do utilizador**: página de visualização com *player* (*play*/pausa, *mute*, volume, ecrã inteiro, seleção de qualidade), com um menu de doações definido pelo criador.
- **Garantia a privacidade do Criador**: capacidade de controlar o acesso à câmara e ao microfone.
- **Implementação do chat em direto**, permite enviar mensagens para cada utilizador durante a transmissão, incluindo o criador.

Estes foram os objetivos traçados para este projeto, que serviu para integrar o Minimum Viable Product (MVP)

1.2.2 Abordagem

Para realizar os objetivos deste projeto a empresa incutiu uma metodologia ágil, que facilita a comunicação e a resolução de problemas em equipa. A metodologia **SCRUM** é considerado uma *framework* de desenvolvimento de *software*.

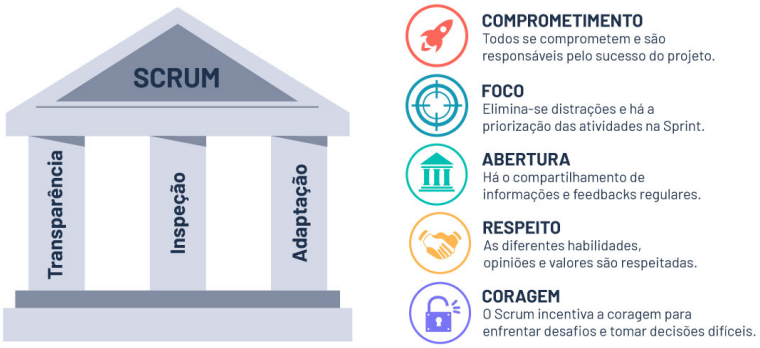

Figura 1.1: Pilares e Valores Da Metodologia SCRUM [2]

Para o projeto foi essencial seguir os pilares da metodologia: **Transparecia**, **Reflexão** (Inspeção), e **Adaptação**. A metodologia não serve apenas e só para a coordenação do projeto, mas os seus valores devem ser seguidos, independentemente do local de trabalho.

"Scrum is an incremental and iterative agile software development framework for managing product development (...) This strategy challenges assumptions of the "traditional, sequential approach"to product development, and encourages teams to self-organize by encouraging close online collaboration of all team members, as well as face-to-face communication among all team members and disciplines in the project.(...)"[3]

A implementação do **SCRUM** no projeto para além do descrito, foi feita através de *Sprints*. Os sprints são espaços de tempo definidos para mostrar as evoluções ao cliente do projeto a desenvolver. Para além disso foram realizadas *daily meetings* que permitiram verificar o processo de cada membro da equipa e avaliar o progresso do projeto como um todo.

1.2.3 Contributos

Tecnológicos: Entrega de uma solução de *streaming* ligado a um controlo de acesso por *tiers*, um *player* simples e responsivo, com *chat* de baixa latência. Adoção de práticas Continuous Delivery/Continuous deployment (CI/CD) e controlo de versões.

Organizacionais. Redução da dependência de ferramentas externas e consolidação da experiência de *streaming* na própria plataforma, aumentando a capacidade de controlo e a qualidade da interação do criador com a sua comunidade.

Pedagógicos. Consolidação de competências em desenho arquitetural, integração de componentes em tempo real e entrega orientada a MVP, com documentação e avaliação objetiva do resultado.

Síntese dos contributos

- Criação de uma plataforma de *streaming* com canais exclusivos por criador.
- Controlo a canais exclusivos por *tiers* e integração com subscrições e pagamentos.
- Disponibilização de um *player* responsivo e simples (play/pausa, volume, ecrã inteiro, seleção de qualidade).
- *Chat* de baixa latência, com texto e emojis, suportado por *API WebSocket*.
- Mecanismos de privacidade e segurança
- Adoção de boas práticas de engenharia (CI/CD, controlo de versões com uso de *branching*, testes automáticos...).
- Crescimento técnico e profissional da individual e da equipa, com aplicação prática de tecnologias não abordadas em contexto académico.

1.2.4 Planeamento do trabalho

Durante o estágio, o desenvolvimento da solução decorreu em etapas bem definidas. Numa fase inicial, realizou-se uma pesquisa com duração aproximada de duas semanas. Concluída essa etapa, a equipa avançou para o planeamento de Use Case (UC) e User Story (US), formalizando critérios e dependências. Seguiu-se a fase de arquitetura, durante a qual foram elaboradas as vistas lógicas, de implementação e físicas, os diagramas de sequência, o modelo de domínio e de dados, bem como a seleção das tecnologias e dos sistemas de gestão de bases de dados. Esta fase inicial estendeu-se por cerca de quatro semanas. Por fim, o supervisor organizou o trabalho em *sprints* de duas semanas no Jira, distribuindo as tarefas até à entrega da solução.

Durante os *sprints*, a equipa seguiu o formato *SCRUM* com *daily meetings*. A implementação foi organizada em fases sequenciais, a partir do desenho produzido na fase de arquitetura, criou-se a camada de domínio e a base de dados. Em seguida, desenvolveram-se as operações Create, Read, Update and Delete (CRUD) e os serviços nucleares de *back-end*, prosseguiu-se com a interface de utilizador e de criador, bem como as integrações, por fim, construíram-se as funções *AWS Lambda* e efetuou-se a ligação *front-end*/*back-end*, finalizado em afinações e testes do MVP. A Tabela 1.2 sintetiza esta distribuição temporal.

Tabela 1.2: Divisão de tempo no projeto

Período de Tempo	**Tarefa**
24/02 a 07/03	Período de pesquisa
10/03 a 21/03	Fase de arquitetura
24/03 a 04/04	**Sprint 1** - Criação das classes de domínio, bases de dados, e preparar o ambiente local & Inicio da pesquisa para o relatório
07/04 a 18/04	**Sprint 2** - Desenvolvimento do CRUD
05/05 a 16/05	**Sprint 3** - Desenvolvimento dos serviços principais de *Back-End* & Realização dos estado da arte
19/05 a 30/05	**Sprint 5** - Desenvolvimento da interface de utilizador e criador
02/06 a 13/06	**Sprint 6** - Criação das lambdas e ligação entre o Front-End e Back-End
16/06 a 22/06	**Sprint 7** - Afinações e testes para o MVP
23/06 a 10/09	Continuação da realização do relatório.

1.3 Estrutura do relatório

Para além da introdução, este documento contém mais 4 capítulos, que apresentam as várias etapas do desenvolvimento da solução.

No capítulo 2, é descrito o estado da arte e são apresentados trabalhos relacionados. No capítulo 3, o planeamento e o desenho, que foram previamente concebidos com a equipa para um melhor enquadramento no projeto como um todo. É realçado o domínio bem como os requesitos funcionais e não funcionais No capítulo 4, são detalhados os vários processos de implementação da solução, onde constam exemplos de funcionalidades, testes para a validação do projeto e imagens com a interface de utilizador. Por fim, o capítulo 5, concentra-se nos resultados obtidos, e numa introspeção final da solução e do relatório.

Capítulo 2

Estado da Arte

Para uma solução mais eficiente e prática, surgiu uma necessidade no ambiente empresarial que levou à pesquisa de diversas tecnologias.

No contexto em que o problema está descrito, é importante procurar aplicações e soluções desenvolvidas por outras empresas e verificar as funcionalidades básicas que estas disponibilizam aos utilizadores.

Neste capítulo, o leitor irá observar as principais funcionalidades que outras aplicações disponibilizam e de que forma contribuíram para o desenvolvimento da solução.

2.1 Trabalhos relacionados

Nesta secção estão apresentadas 5 empresas que têm como uma *features* da sua aplicação o *streaming*. Servirá para identificar funcionalidades que auxiliaram o projeto, bem como as funcionalidades discutas com a empresa.

Ao longo dos subcapítulos, vão ser explicados os diferentes protocolos de transmissão em direto, usados pelas diferentes empresas, que após discussão com a equipa foram escolhidos com base na sua latência e na sua complexidade.

2.1.1 Youtube

YouTube Live tem diversas Application Programming Interface (API) oficiais em Java, Python, Node.js, Go, Ruby e PHP, usadas para automatizar a programação de *lives*, *chat*, e metadados. A ingestão de vídeo suporta Real-Time Messaging Protocol (RTMP) e também HTTP Live Streaming (HLS) para *push* a partir de *encoders* compatíveis. O reprodução ao utilizador é feita sobre HLS, com diversos tipos de latência.[4]

Entre as principais funcionalidades desta plataforma encontrou-se:

- Métricas em tempo real como: perdas de *frames*, *bitrate*, alertas de estabilidade.
- Pausa e retorno durante a transmissão.
- Moderação do chat
- Legendas automáticas e em tempo real

A interface gráfica da é intuitiva, responsiva e moderna. Permite, assim, ao criador uma navegação eficiente e acessível pelas funcionalidades da aplicação.

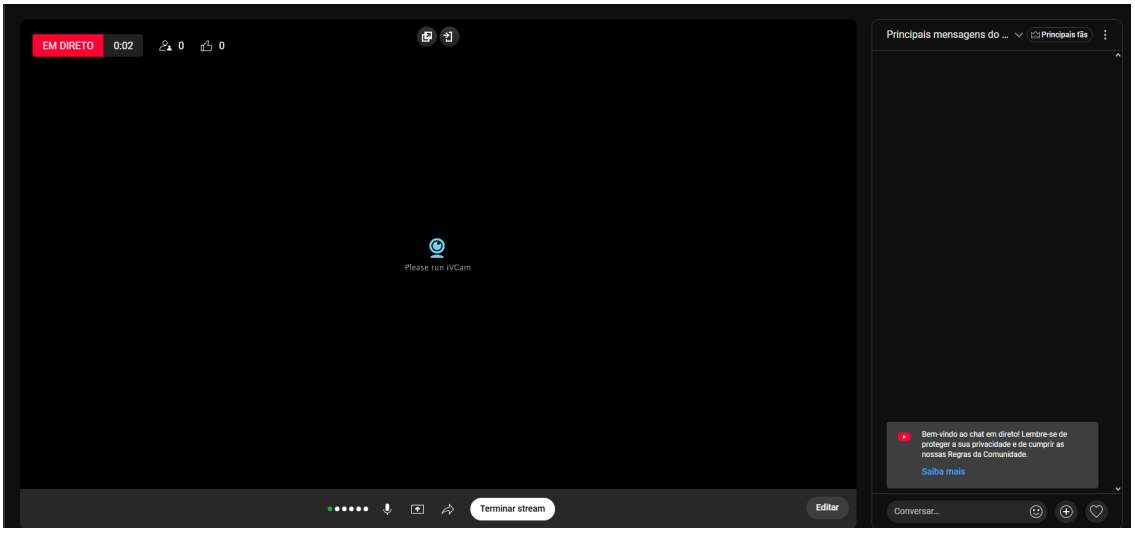

Figura 2.1: Interface de Criador de Conteúdo do Youtube Live **Youtube**

A escolha do microfone, câmara, titulo e descrição são efetuadas nas etapas antes de iniciar a transmissão. Para além destas etapas a plataforma permite a mudança destas funcionalidades durante a transmissão.

A *livestream* pode ser efetuada em diversos dispositivos, com etapas semelhantes, e com diferentes programas que facilitam a transmissão de conteúdo.

Pontos Fortes

- Alcance e robustez de infraestrutura.
- Transcodificação automática para múltiplas qualidades (*bitrate*).
- Ferramentas nativas de monetização (anúncios, *Super Chat*/*Stickers*, *Memberships*).
- Analítica integrada em tempo real.

Pontos fracos

- Navegação inicial mais longa, com um excesso de menus para o criador.
- Inserção de anúncios pode interromper o ritmo do *chat* se usada sem planeamento.
- Títulos, restrições e visibilidades exigem *checklist* para evitar deslizes.
- Difícil expansão dentro da plataforma.

2.1.2 KICK

As integrações disponibilizadas pela **Kick** assentam numa API pública REST/JSON, com endpoints para canais, utilizadores, *chat*, moderação e *livestreams*. O *ingest* de vídeo para criadores é realizado por RTMP/RTMPS configuradas num *encoder* (ex: OBS).

A plataforma disponibiliza as principais funcionalidades:

- Creator Dashboard.

- *Replays* automáticos após terminar a transmissão.
- Monetização mais abrangente.
- *Co-stream* nativa.

Numa perspetiva de visualizador, a plataforma apresenta uma interface focada especialmente no vídeo. A interação com a comunidade é feita via *low latency chat*, com emojis e difetentes cores para cada utilizador.

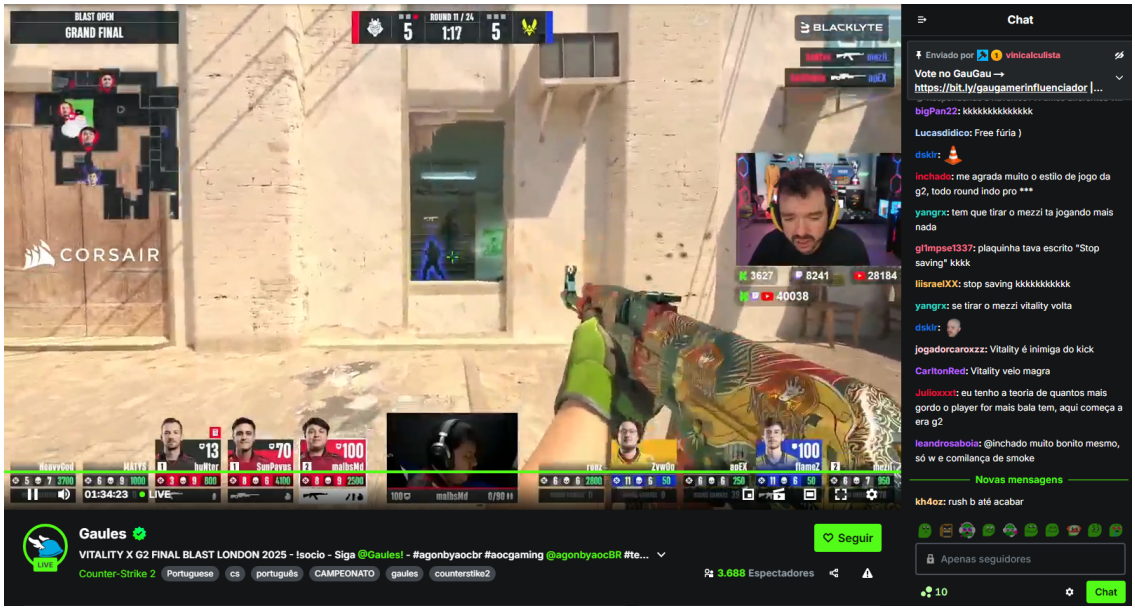

Figura 2.2: Interface de utilizador da Kick [5]

A configuração de utilizador é pouca o que permite assistir a uma transmissão ao vivo rapidamente, além disso, o utilizador tem diversas maneiras de interagir com o criador, podendo tirar excertos da *stream*, paritcipar em sondagens, doar, e mandar mensagem privada ao mesmo.

Pontos fortes

- Fluxo de arranque simples.
- Ferramentas de moderação claras.
- Condições de monetização competitivas.

Pontos fracos

- Ecossistema de extensões e API ainda em evolução.
- Dependência de serviços externos.
- Documentação pública de protocolos de entrega pouco detalhada.

2.1.3 Twitch

A **Twitch Live** foi apresentado pela **Twitch Interactive** em 2011, após a aquisição pela **Amazon** em 2014[6]. A **Twitch** expõe uma **API REST**, *webhooks* e *EventSub* via *WebSocket*. O principal ecossistema usa *JavaScript*/*TypeScript* para o chat e *Java*/*Kotlin*, C#, Python e Go para as restantes API's. Durante a transmissão o *ingest* é feito por RTMP a entrega ao público é por HLS.

As principais características da plataforma de *streaming* são:

- Painéis personalizáveis para o estado do stream, *chat*, alertas, *stream markers*...
- Baixa latência.
- Co-stream nativo
- Monetização integrada.
- Extensões personalizáveis para melhorar a analítica da transmissão.

A interface embora interativa e personalizável, dispões de uma poluição visual grande, existem imensas definições e características que numa primeira interação podem desmotivar o criador a iniciar a transmissão.

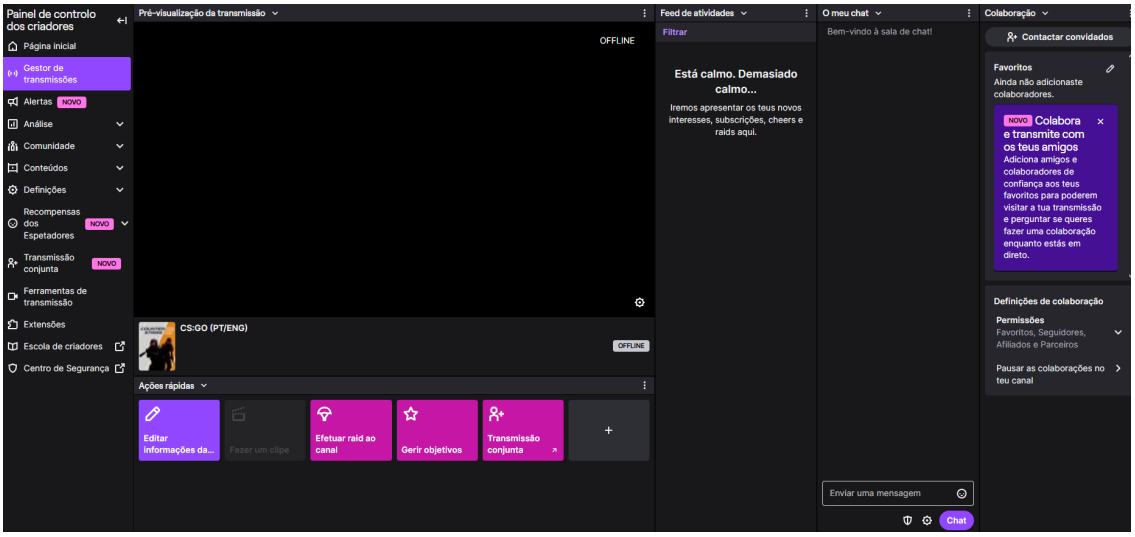

Figura 2.3: Interface de Criador de Conteúdo da Twitch [7]

Contudo, a disposição de menus e opções descritivas ajudam o criador mais técnico a personalizar o seu ambiente e a uma melhor interação com a sua comunidade.

Pontos fortes

- Painel de transmissão personalizável
- Grande número de ferramentas de moderação.
- As extensões permitem uma grande personalização do ecossistema.
- Padrões de interação rápida como por exemplo: chat, *emojis* e criação de excertos da transmissão pelos utilizadores.

Pontos fracos

- *Dashboard* denso.
- Monetização com regras restritas.
- Gestão de anúncios sensível, pois podem colidir com momentos importantes do conteúdo.
- Contínua mudança de nomenclatura das definições.

2.1.4 TikTok

O TikTok disponibiliza diferentes SDK's para Android, IOS e Web, existem ainda *Webhooks* e *APIs* para integrações do lado do servidor.

Para o *streaming*, o *ingest* do ambiente é feito por RTMP/RTMPS ou por algum *encoder* externo. A entrega aos utilizadores é feita por HLS.

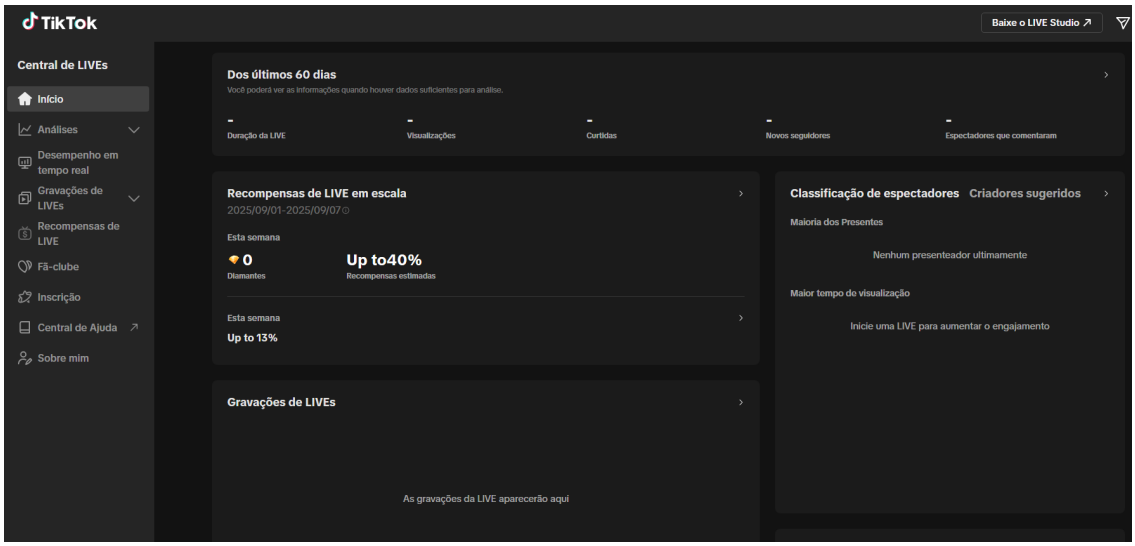

Figura 2.4: *LIVE* center do **TikTok**

Organizado em barra lateral, cabeçalho e grelha de cartões informativos, a interface mostra uma *dashboard* necessária para uma expansão na aplicação, pois todas as transmissões são personalizáveis em termos de interação e moderação.

O controlo da *livestream* está totalmente dependente do utilizador, mas por outro lado todos os dados da transmissão são disponibilizados ao criador, com foco na monetização e transações com uma moeda interna.

Pontos fortes

- Monetização nativa.
- Presentes e efeitos destacam-se e mantêm o público a participar.
- *Co-stream* predefinidos e convites em poucos passos.

Pontos fracos

- Ambiente visual poluído.
- Limita criadores que trabalham noutros sistemas, pois o encoder internos funciona apenas no *Windows*.
- Prioridade ao formato vertical.

2.1.5 Discord

O **Discord Go Live** foi apresentado pela **Discord Inc.** em 2019. O seu desenvolvimento foi feito sobretudo em *JavaScript*/*TypeScript*, *Python*, *Go* e *Java*. Possui sobretudo *API's REST* e *WebSocket's* oficiais. Para voz e vídeo, as ligações usam Real-time Transport Protocol (RTP)/User Datagram Protocol (UDP). Diferente de plataformas anteriormente referidas o *broadcast* não é nativo no cliente, mas sim com retransmissão pelo *backend*.

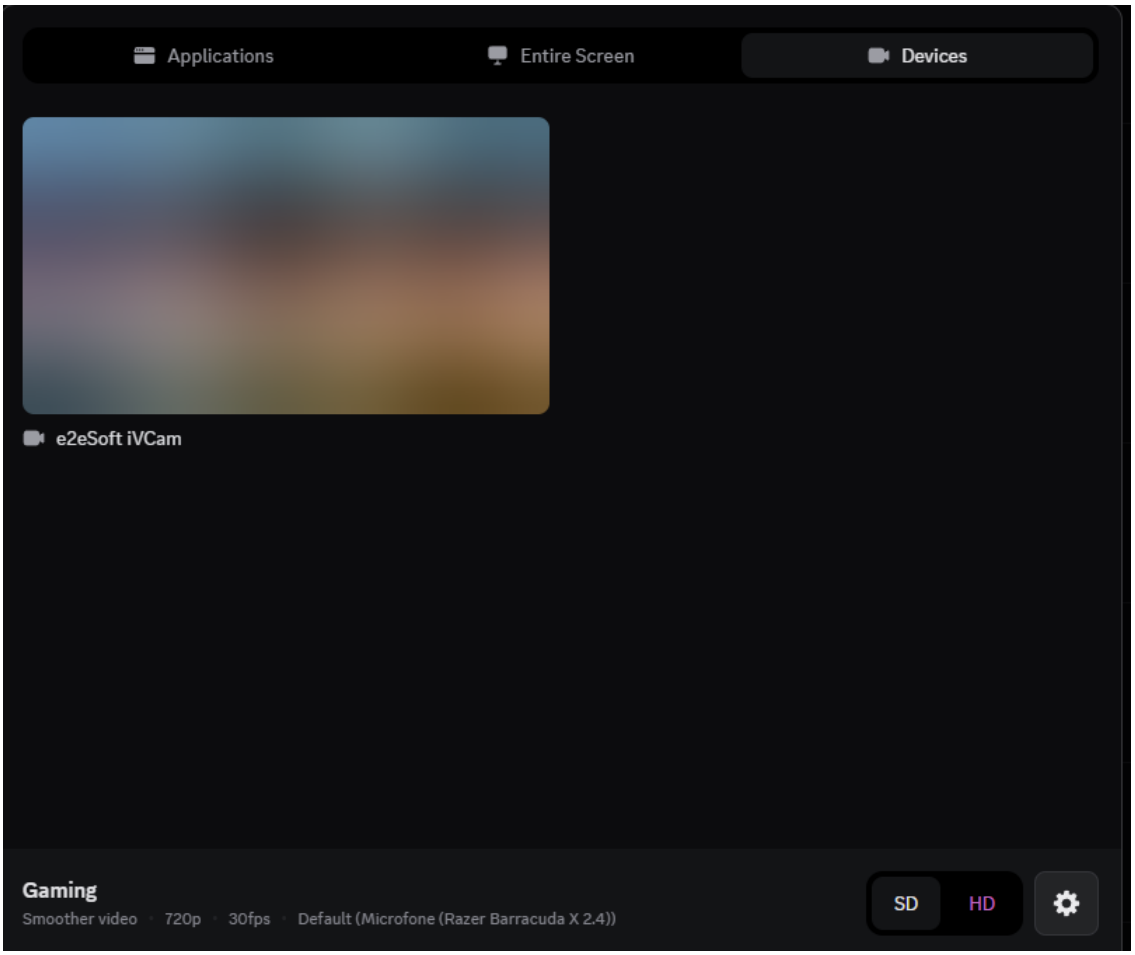

Figura 2.5: Escolha de cenário para transmissão

A interface foca-se na seleção de fonte para a transmissão com três zonas principais.

A barra de separadores, contêm diferentes de secções, em que o utilizador pode escolher entre partilhar alguma aplicação especifica, partilhar o ecrã inteiro ou escolher algum dispositivo para iniciar a transmissão.

O rodapé mostra algumas definições de controlo como o tema, a qualidade e o dispositivo de áudio. Este controlo dado pela plataforma mostra que nao é necessario alguma configuração por parte do utilizador para definidir tanto os dispositivos de entrada como os de saída.

Pontos fortes

- Partilha de ecrã sem assistentes longos.
- Vídeo, voz e *chat* lado a lado.
- Partilha de vídeo privada.

Pontos fracos

- Como a transmissão é só feita em "chamada"não existe crescimento na plataforma.
- A qualidade depende muito de transações internas
- Alguma partilha de conteúdo é bloqueada.

2.1.6 Tabela Comparativa das Plataformas

Para uma melhor visualização das diferentes características das aplicações construiu-se a seguinte tabela:

Tabela 2.1: Tabela Comparativa das Plataformas

Funcionalidade / Característica	**YouTube**	**Kick**	**Twitch**	**TikTok**	**Discord**
Início de transmissão intuitivo		x		x	x
Utilização de programas externo para a transmissão	x	x	x	x	
Moderação de *chat*	x	x	x	x	x
Monetização nativa em direto	x	x	x	x	
Replay da transmissão	x	x	x	x	
Analítica em tempo real e pós-live	x	x	x	x	
Interação em tempo quase real (boa latência)	x	x	x	x	x
Painel do criador unificado (operar tudo num ecrã)	x	x	x	x	
Extensibilidade / overlays prontos na plataforma			x		
Experiência otimizada na *vertical*				x	
Experiência otimizada na *landscape*	x	x	x		x
Alcance além de seguidores	x	x	x	x	
Adequado a comunidades privadas					x

A Tabela 2.1 mostra três plataformas equivalentes: **YouTube**, **KICK** e **Twitch**, pois oferecem uma experiência de *broadcast* completa (monetização, *replays*, analítica e painéis de criador complexos). O **TikTok** privilegia rapidez e interação social nos dispositivos *mobile*.

O **Discord** aposta em simplicidade e colaboração em espaços privados, prescindindo de replays e monetização nativa. Em termos de moderação e *chat*, as quatro plataformas de *streaming* mantêm ferramentas complexas, por outro lado, o **Discord**, foca-se na conversa em tempo real dentro do próprio canal.

2.2 Tecnologias existentes

Após análise das tecnologias utilizadas nas soluções das diversas empresas e com as discussões em grupo com a equipa foi alcançado o conhecimento base e necessário para perceber a melhor abordagem a seguir.

2.2.1 Protocolos De Streaming

Os protocolos de *streaming* estruturam o percurso do sinal entre captura, *ingest*, codificação e entrega ao público, e dependendo do protocolo pode implicar a latência e compatibilidade. A Tabela 2.2 compara os protocolos mais relevantes (RTMP/RTMPS, HLS/LL-HLS, WebRTC e MediaStream) quanto a uso típico, vantagens e limitações.

Tabela 2.2: Resumo comparativo de protocolos de *streaming*

Protocolo	**Uso típico**	**Pontos fortes**	**Pontos fracos**
RTMP/RTMPS [8]	*Ingest* do criador	Configuração simples e uma alta compatibilidade com diversos *enconders*.	Sendo um dos primeiros protocolos tem uma latência maior.
HLS / LL-HLS [9]	Entrega ao utilizador	Tem um amplo suporte tanto em *web* como em *mobile*, a versão de baixa latência melhoras o desempenho do HLS clássico	O HLS clássico apresenta uma latência considerável (15-30), LL-HLS pede mais poder computacional mas entrega com pouco *delay*
WebRTC [10]	Interatividade e baixa latência	Um dos protocolos com a menor latência, bidirecional e nativo do *browser*.	O escalonamento é condicionado pelos custos elevados.
MediaStream	Captura local no *browser*	Nativo nos *browsers* modernos, acesso direto a câmara e microfone.	Requer combinação com WebRTC/RTMP para *ingest* e dependente de permissões do utilizador e políticas de segurança do *browser*.

2.2.2 Back-End

No *back-end*, as decisões impactam o desempenho das APIs, a fiabilidade dos serviços e a escalabilidade de fluxos críticos (autenticação, gestão de sessões, monetização e VOD). A Tabela 2.3 resume as lingaguem correntes, com ênfase em modelos arquiteturais.

Tabela 2.3: Tabela resumo das tecnologias *back-end* (BE)

Linguagem / Framework	**Arquitetura**	**Pontos fortes**	**Pontos fracos**
Node.js (Express)	REST/GraphQL, microserviços	Mesma linguagem do FE, NPM vasto, produtividade alta.	*CPU-bound* menos eficiente, *package* com eficiencia desigual.
Python (FastAPI)	Comaptibilidade REST, microserviços	Desenvolvimento rápido, boa documentação, desempenho adequado.	Limita as tarefas *CPU-Bound*, dependências podem crescer.
Java (Spring Boot)	Monólito/modular, microserviços	Robustez empresarial, ecossistema amplo	Consumo de memória e *startup* maiores, curva de aprendizagem.
Go	Serviços leves, REST	Concorrência simples e baixo uso de recursos.	Ecossistema web menos abrangente.
PHP	Monolítica	Compatibilidade elevada e facil integração com o Front-End	Falta de padronização e de continuidade

2.2.3 Front-End

A escolha da tecnologia *front-end* condiciona desempenho e experiência do utilizador, live dashboards. Na Tabela 2.4 sintetizam-se as opções mais usadas, destacando arquitetura, pontos fortes e limitações práticas relevantes para produtos de vídeo em direto.

Tabela 2.4: Tabela resumo das tecnologias *front-end* (FE)

Linguagem / Framework	**Arquitetura**	**Pontos fortes**	**Pontos fracos**
React	Single Page Application (SPA); Server Side Rendering (SSR) via Next.js	Responsivo, forte reutilização de componentes, comunidade ampla, boa produtividade.	Muitas escolhas de bibliotecas, risco de inconsistências visuais, requer organização em projetos maiores.
Next.js (React)	SSR/ISR; *routing* e *data fetching* integrados	Bom desempenho, arranque simples, páginas e rotas prontas.	Convenções próprias, atualizações podem exigir ajustes, alguma rigidez fora do padrão.
Vue	SPA; SSR com Nuxt	Aprendizagem rápida, componentes claros, documentação acessível, curva suave.	Menor adoção corporativa, ecossistema mais reduzido, estilos de implementação podem variar entre equipas.
Angular	SPA; SSR com *Angular Universal*	Padrões consistentes, boas ferramentas oficiais, adequado a equipas grandes.	Curva de aprendizagem longa, pode tornar-se pesado e menos flexível.

2.2.4 Base de Dados

A camada de dados sustenta contas, conteúdos, transações e métricas de audiência, exigindo combinações de armazenamento transacional e serviços orientados a desempenho. A

Tabela 2.5 apresenta SGBD amplamente adotados, evidenciando modelos, pontos fortes e fragilidades para cenários de live/VOD.

Tabela 2.5: SGBD e tecnologias de dados mais comuns

SGBD / Tecnologia	**Modelo**	**Pontos fortes**	**Pontos fracos**
PostgreSQL	Relacional (ACID)	Muito completo e estável, extensível, bom suporte a dados modernos.	Exige afinação em projetos grandes, escala horizontal mais trabalhosa.
MySQL/MariaDB	Relacional (ACID)	Simples de gerir, rápido em leituras, comunidade ampla.	Menos recursos avançados, pode exigir soluções de contorno em cenários complexos.
MongoDB	Documento (NoSQL)	Esquema flexível, evolução rápida, adequado a dados heterogéneos.	Consistência e consultas complexas requerem desenho cuidadoso.
Redis	Chave–valor / *cache*	Muito rápido, excelente para *cache*, sessões e filas.	Persistência limitada, gestão de memória crítica, não substitui a base principal.

2.3 Análise e seleção das tecnologias

Após análise das tecnologias utilizadas nas plataformas e com as *daily meetings* com a equipa, alcançou-se o conhecimento necessário para perceber a melhor abordagem a seguir.

2.3.1 Protocolos de Streaming

Num enquadramento geral, opta-se pela captura nativa no navegador, *ingest* em baixa latência com WebRTC através do Interactive Video Service (IVS) Web Broadcast SDK da AWS e distribuição por HLS/LL-HLS para maximizar compatibilidade. A escalabilidade tira partido da infraestrutura global da AWS IVS (*ingest points*, transcodificação e *playback*).

Para a captura, o MediaStream, suportado nativamente nos *browsers* modernos permite o acesso controlado a câmara/microfone. Tal reduz complexidade ao cliente e mantém a privacidade por defeito.

Utilizou-se WebRTC via IVS Web Broadcast SDK para atingir latências muito baixas na ordem de um a três segundos. A escolha evita a manutenção de servidores próprios, delegando essa preocupação com os serviços da AWS. A presença de streamKey e ingestEndpoint garante isolamento por canal.

A reprodução é servida em HLS (com LL-HLS ativado no player via hls.js), o que assegura uma compatibilidade ampla tanto em *web* como em *mobile*. O IVS trata da codificação e expõe um *playbackUrl*. Esta decisão equilibra latência reduzida com robustez, tolerância a variações de rede e escala global.

2.3.2 Back-end

Após a pesquisa ponderada, a equipa decidiu escolher como tecnologia principal o **Node.js** com *Express*, a linguagem principal é o JavaScript, o que torna quaisquer operações Input/Output (I/O) rápidas e leves. Para uma plataforma de **social media** é necessário existir uma ligação com os casos de uso e com a tecnologia escolhida, para tal sabemos que um dos objetivos principais é a baixa latência e a transmissão em tempo real, logo o Node.js foi a tecnologia que a equipa se inclinou desde inicio.

Para além disso, o **Express** fornece um *framework* minimalista, com *routing* e *middleware* (autenticação, *rate limiting*, CORS), o que acelera a implementação de *APIs* estáveis e testáveis.

A pilha **JavaScript** facilita a integração com SDKs AWS e suporta adoção progressiva de **TypeScript** para maior segurança estática.

2.3.3 Front-end

Para o desenvolvimento do front-end, existia uma indecisão em relação a duas tecnologias, uma delas nunca trabalhada pela equipa e outra com conhecimento prévio pela mesma.

- Biblioteca **React** do **JavaScript**
- Framework **Angular**

A Tabela 2.6 ajuda a comparar as duas abordagens

Tabela 2.6: Comparação entre React e Angular

Funcionalidade/ Critério	**React**	**Angular**
Linguagem Base	JavaScript com HTLM	TypeScript
Casos de uso	SPA, Modularização, *dashboards*	Grande Aplicações estruturadas
Curva de aprendizagem	Elevada	Moderada e flexível
Tipo	Biblioteca de JavaScript	Framework completo

A equipa decidiu escolher o **React** porque, ao combinar React, Router e JSX no cliente com Node.js/Express no servidor, resulta uma arquitetura coesa e preparada para ambientes em tempo real.

2.3.4 Base de dados

A equipa optou por SQL (Aurora PostgreSQL) para dados sensíveis, críticos e com forte exigência de integridade e segurança como: ids, *usernames*, emails .... E por NoSQL (DynamoDB) para dados altamente editáveis e de apresentação como: preço, título, descrição, flags e preferências. Esta divisão maximiza coerência transacional onde é necessária e flexibilidade onde é benéfica.

2.4 Conclusão

Conclui-se que o ecossistema de *livestreaming* organiza-se em três partes: transmissão pública (YouTube, Twitch, TikTok/Kick), *mobile* (TikTok) e colaboração privada (Discord).

A decisão deve privilegiar usabilidade do painel do criador, monetização e reaproveitamento (VOD/clips).

Em termos técnicos, prevalece RTMP para ingest, HLS/LL-HLS para entrega e WebRTC quando a latência sub-segundo é crítica. No stack, FE modernos (React/Next, Vue, Angular) e BE leves (Node.js+Express) oferecem rapidez e manutenção aceitável.

Para dados, PostgreSQL e MySQL servem o núcleo transacional, com MongoDB/Redis a complementar flexibilidade e desempenho.

Capítulo 3

Análise e desenho da solução

O capítulo que se segue tem o principal objetivo de apresentar as diferentes fases ligadas ao processo de análise e desenho da solução. Inicia-se com a contextualização do domínio do problema, suportada por um modelo de domínio e por uma tabela de termos. Procede-se à especificação dos requisitos funcionais e não funcionais, antecedendo a identificação dos atores e dos casos de uso. Por último, descreve-se o desenho da solução, evidenciando os principais componentes e a arquitetura da aplicação.

3.1 Domínio do problema

Caracteriza-se o contexto operacional da plataforma **OnlyYes** e o núcleo do problema a resolver descrito na secção 1.2.

Esta secção estabelece o vocabulário comum e as relações conceptuais que suportam o desenho e a implementação da solução, e o modelo que caracteriza as regras de negocio.

3.1.1 Glossário do Domínio

O glossário acrescenta significado ao domínio, pois contêm termos fundamentais para clarificar retirar ambiguidades.

A Tabela 3.1 apresenta o termo do domínio em inglês e a sua tradução em portuguesa para melhor compreensão, na última coluna da tabela são explicados os termos no contexto do domínio

Tabela 3.1: Glossário do domínio da aplicação

Termo (EN)	**Termo (PT)**	**Descrição**
Account	Conta	Entidade que representa o utilizador
Profile	Perfil	Entidade que representa a identidade do utilizador na aplicação
Livestream	Transmissão em direto	Entidade que representa a emissão de vídeo e de áudio em tempo real.
Livestream Info	Parâmetros de transmissão	Entidade que representa o conjunto de parâmetros técnicos necessários para iniciar e manter a transmissão
Tier	Escalão	Nível de acesso com preço e benefícios definidos.
Live Menu	Menu da transmissão	Conjunto de ofertas ou ações disponíveis durante a transmissão.
Live Menu Item	Item de menu	Elemento individual do menu com nome, preço e disponibilidade.
Email	Email	Endereço eletrónico associado à conta.
Username	Nome de utilizador	Identificador da conta.
Status	Estado	Situação atual de uma subscrição
Title	Título	Texto curto que identifica um conteúdo.
Description	Descrição	Texto que explica ou contextualiza um conteúdo.
streamKey	Chave de transmissão	Código utilizado para autorizar o envio do sinal de transmissão.
Channel	Canal	Unidade lógica onde as transmissões são realizadas.
Ingest Endpoint	Endereço de ingestão	Endereço utilizado para enviar o sinal de transmissão para o serviço.
Playback Url	Endereço de reprodução	Endereço utilizado para visualizar o conteúdo de uma transmissão.
Perks	Benefícios	Conjunto de vantagens incluídas num escalão.

Os temos da tabela não representam todas a nomenclaturas usadas na documentação, apenas as mais específicas e que podem gerar alguma confusão.

3.1.2 Modelo de Domínio

Na figura 3.1, está apresentadoaw o diagrama conceptual Unified Modeling Language (UML) que representa entidades e relações principais.

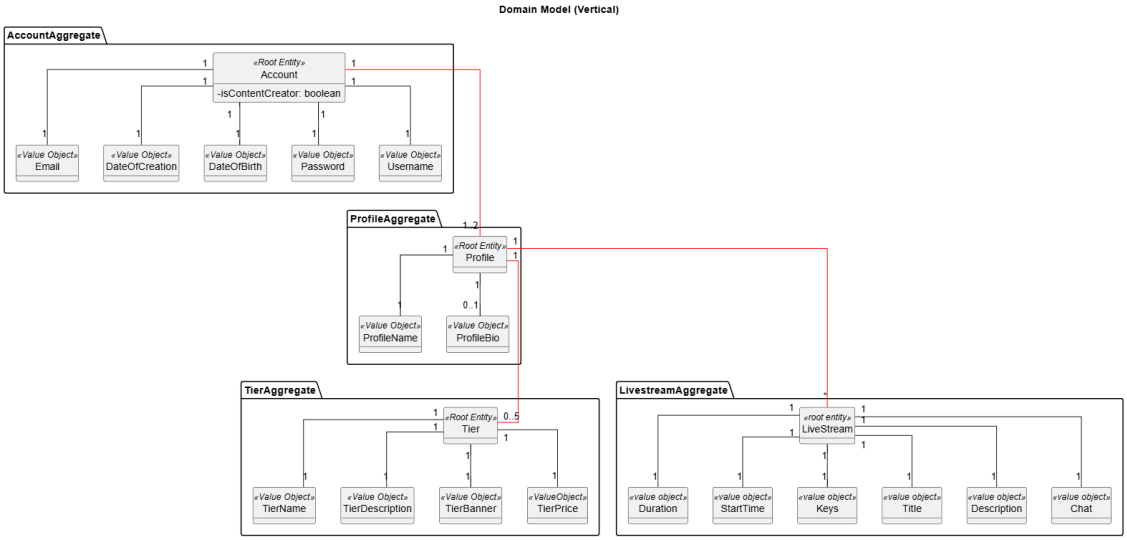

Figura 3.1: Modelo de Dominio - Streaming

A solução é composta por quatro agregados principais: *Account*, *Profile*, *Tier*, *Livestream*. Cada agregado define e representa uma área funcional do domínio, composta pela sua entidade e pelos seus objetos.

As ligações entre agregados são as seguintes:

O agregado *Account* é responsável pela integridade dos utilizadores, composta por identificadores, *email* e *username*, e por objetos que servem apenas para conformidades e para fornecer uma melhor experiência ao utilizador. O agregado *Account* liga-se ao agregado *profile* numa ligação de um para muitos, mas no caso do domínio a sua multiplicidade será de uma para um ou dois.

O agregado *Profile* representa a identidade visual do utilizador na aplicação. Um utilizador pode ter um ou dois perfis, sendo o primeiro publico e com acesso a todos os utilizadores e o segundo só permitido a utilizadores que tenham o escalão definido pelo utilizador. Após a criação do perfil privado o sistema permite ao utilizador criar o seu ambiente de transmissão e o seu ambiente de escalões.

O agregado *Tier* modela os níveis de acesso e benefícios por perfil. Contém *TierName*, *TierDescription*, *TierBanner* e *TierPrice*, impondo as seguintes invariantes: no máximo cinco *tiers* por *Profile*.

O agregado *LiveStream* encapsula o ciclo de vida de uma transmissão em direto, incluindo *Title*, *Description*, *StartTime*, *Duration*, *Keys* e *Chat*. Este agregado é dependente da criação de um perfil privado.

3.2 Requisitos Funcionais e não funcionais

Para continuar o desenho da solução, foi necessário declarar e especificar os requisitos que a aplicação deve cumprir. Os requisitos apresentados foram definidos nas *daily meetings* com a equipa e validados pelo supervisor.

Nesta secção vão ser apresentados dois tipos de requisitos:

- Requisitos funcionais
- Requisitos não funcionais

3.2.1 Requisitos Funcionais

Um requisito funcional clarifica no âmbito funcional, as restrições e os critérios de qualidade a que a aplicação deve obedecer.

"Functional requirements capture the intended behavior of the system. This behavior may be expressed as services, tasks or functions the system is required to perform."[11]

Tabela 3.2: Requisitos Funcionais

Nº	**Descrição do Requisito**
RF01	O sistema deve permitir iniciar uma transmissão de forma intuitiva e rápida.
RF02	O sistema deve permitir encerrar uma transmissão de forma intuitiva e rápida.
RF03	O sistema deve bloquear o acesso a visualizadores que não possuam o *tier* adequado para visualizar uma transmissão.
RF04	O sistema deve permitir ao criador o controlo do título, descrição, câmara, microfone e cena.
RF05	O sistema deve confirmar que o criador tem perfil privado para criar uma transmissão em direto.
RF06	O sistema deve permitir que o criador personalize o menu interativo da transmissão.
RF07	O sistema deve usar o serviço de *streaming* da AWS para melhor monitorização.
RF08	O sistema deve permitir interação entre criador e visualizadores através de um *chat* em tempo real.

3.2.2 Requisitos Não Funcionais

Nesta secção estabelece-se o enquadramento dos requisitos não funcionais e do modelo FURPS+ ligando-os aos requisitos funcionais.

"Non-functional requirements help deliver a positive user experience and ensure that product performs to the standards set by stakeholders, clients, and customers."[12]

FURPS+

O modelo Functionality, Usability, Reliability, Performance,Supportability (FURPS+), serve para classificar em atributos de qualidade um softwate [12]:

- *Functionality*: Especificações funcionais que, em geral, representam as principais funcionalidades da solução.
- *Usability*: Considera aspetos da interface do utilizador, como estética e consistência.
- *Reliability*:Característica que está associada à inoperabilidade, recuperação em caso de falhas e integridade da aplicação.
- *Performance*: Refere-se aos tempos de resposta, eficiência, escalabilidade e utilização dos recursos da aplicação.
- *Supportability*: Característica relacionada com testes, compatibilidade e adaptabilidade da aplicação.

No sistema:

Functionality

- Iniciar e terminar transmissão em dois cliques (RF01/RF02)
- Autorização por escalão antes de emitir a transmissão (RF03)
- Bloquear criação da transmissão se o perfil não cumprir os critérios. (RF03/RF05)
- Controlos do criador (título, descrição, câmara, microfone, cena) com pré-visualização. (RF04)
- Menu interativo configurável pelo criador com descrição, preço e visibilidade. (RF06)
- Garantir interatividade pelo *chat* (RF08)

Usability

- A interface deve ser intuitiva e consistente, permitindo ao utilizador navegar facilmente entre funcionalidades.
- O sistema deve apresentar mensagens de erro e *feedback* claro sobre o estado das operações.

Reliability

- As validações na criação e atualização de dados devem garantir a integridade da aplicação, evitando operações não autorizadas.

Performance

- O sistema deve permitir uma latência minima de acordo com a capacidade de rede de cada utilizador.
- Não deve haver perda significativa de qualidade durante a transmissão.

Supportability

- A solução deve ser modular, permitindo que os componentes sejam alterados, testados e mantidos de forma independente.
- A solução deve permitir que os utilizadores utilizem diversos navegadores.

+ Scalability

- A estrutura modular deve permitir a integração com novas áreas de negócio sem comprometer o desempenho do sistema e a experiência do utilizador.

3.3 Casos de Uso

Na solução, foram considerados dois atores que fazem parte do sistema, o criador e o visualizador, os casos de uso foram divididos conforme as responsabilidades de cada ator.

Para melhor compreensão e legibilidade do documento os diagramas de caso de uso passaram para os anexos.A.2 A.3

Na Tabela 3.3 estão apresentados os casos de uso do criador:

Tabela 3.3: Casos de uso do ator *Creator* A.2 & A.3

ID	Caso de uso
C1	Definir um titulo para a minha transmissão
C2	Definir uma descrição para a minha transmissão
C3	Pre-Visualizar a minha transmissão
C4	Visualizar a minha transmissão
C5	Iniciar a minha transmissão
C6	Definir a minha câmara
C7	Definir o meu microfone
C8	Silenciar o meu microfone
C9	Ocultar a minha câmara
C10	Terminar a minha transmissão
C11	Visualizar o chat da minha transmissão
C12	Interagir com o chat da minha transmissão
C13	Criar um menu interativo para a minha transmissão
C14	Definir tiers para a minha transmissão

Na Tabela 3.4 estão apresentados os casos de uso do visualizador:

Tabela 3.4: Casos de uso do ator *Viewer* A.1

ID	Caso de uso
V1	Visualizar a transmissão de um Creator
V2	Interagir com o chat da transmissão
V3	Controlar a visualização [play,mute,fullscreen,return to Live]
V4	Definir a qualidade da transmissão

3.4 Desenho

Nesta secção, a solução foi desenhada através do modelo C4, contempla diferentes vistas, o modelo de dados e a relação entre as tabelas. Por fim, foram desenhados os System Sequence Diagram (SSD) para completar o resto da arquitetura.

3.4.1 Modelo C4

O modelo C4, apresentado por Simon Brown no vídeo [13] propõe uma abordagem hierárquica e incremental para comunicar arquiteturas de software, o modelo divise-se em quatro camadas:

- *System Context* - O sistema mais os utilizadores e as suas dependências.
- *Containers* - A forma global da arquitetura e as escolhas tecnológicas.
- *Components* - Os componentes lógicos e as respetivas interações dentro do *container*.
- *Code*- Detalhes de implementação.

3.4.2 Nível 1

Na Figura 3.2 é mostrado a vista mais simples do sistema, apenas com o componente central e os serviços externos ao mesmo.

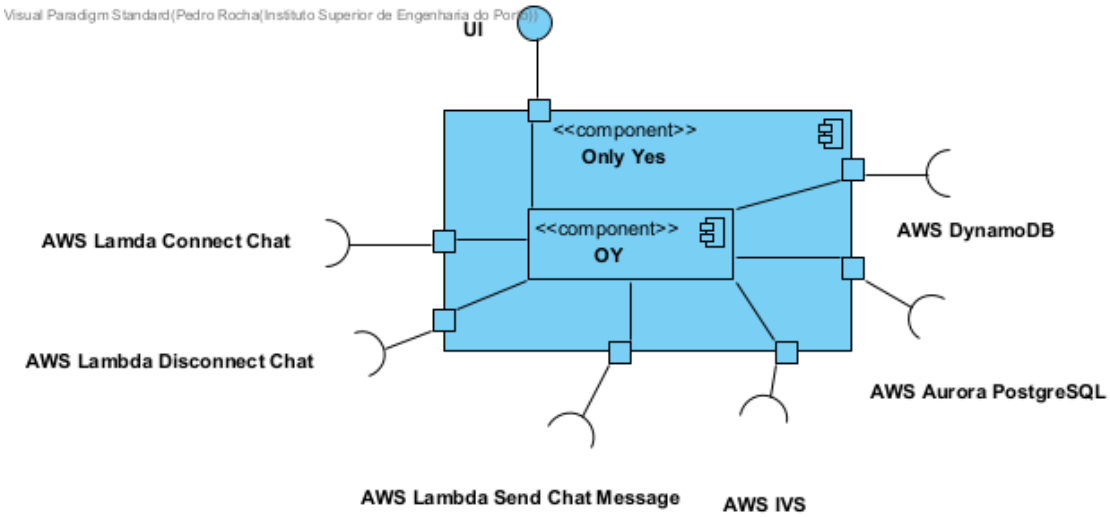

Figura 3.2: Vista Lógica do Sistema - Nível 1

O sistema disponibiliza uma interface a ser consumida pelo utilizador e consome outros serviços, descritos na Tabela 3.5 :

Tabela 3.5: Serviços disponibilizados para o consumo do sistema

Serviço	Descrição
AWS Lambda Connect Chat	Função *serverless* que permite ao utilizador juntar-se a *web-Socket* para receber e enviar mensagens em tempo real.
AWS Lambda Disconect Chat	Função *serverless* que permite ao utilizador sair da *webSocket* e da sala de transmissão.
AWS Lambda Send Chat Message	Função *serverless* que permite ao utilizador enviar mensagens para o *chat* da sala de transmissão.
AWS IVS	Serviço gerido pela AWS que cria os canais de transmissão e as credenciais únicas de cada criador.
AWS Aurora PostgreSQL	Base de dados SQL gerida pela AWS.
AWS DynamoDB	Base de dados NoSQL gerida pela AWS

3.4.3 Nível 2

A Figura 3.3 apresenta-se uma vista que divide o componente principal no *front-end* e no *back-end*, cada um com as suas ligações aos respetivos serviços.

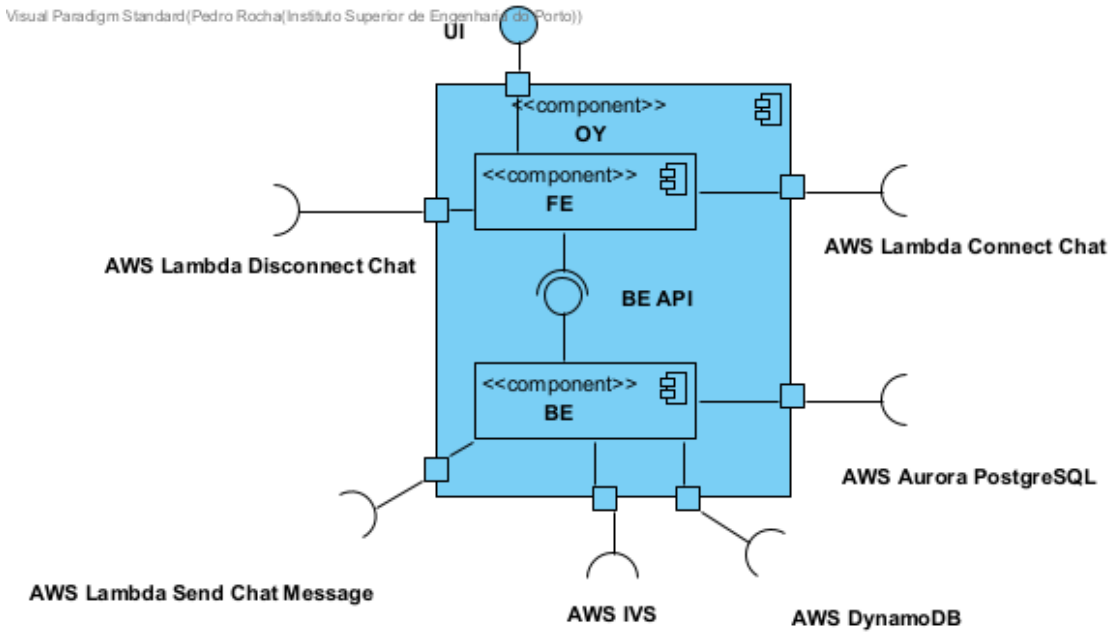

Figura 3.3: Vista Lógica do Sistema - Nível 2

O componente BE(*back-end*) proporciona uma APIREST que o componente FE(*front-end*) consome.

3.4.4 Nível 3

No nível três, deste modelo, começa-se por apresentar a vista lógica do *back-end*.

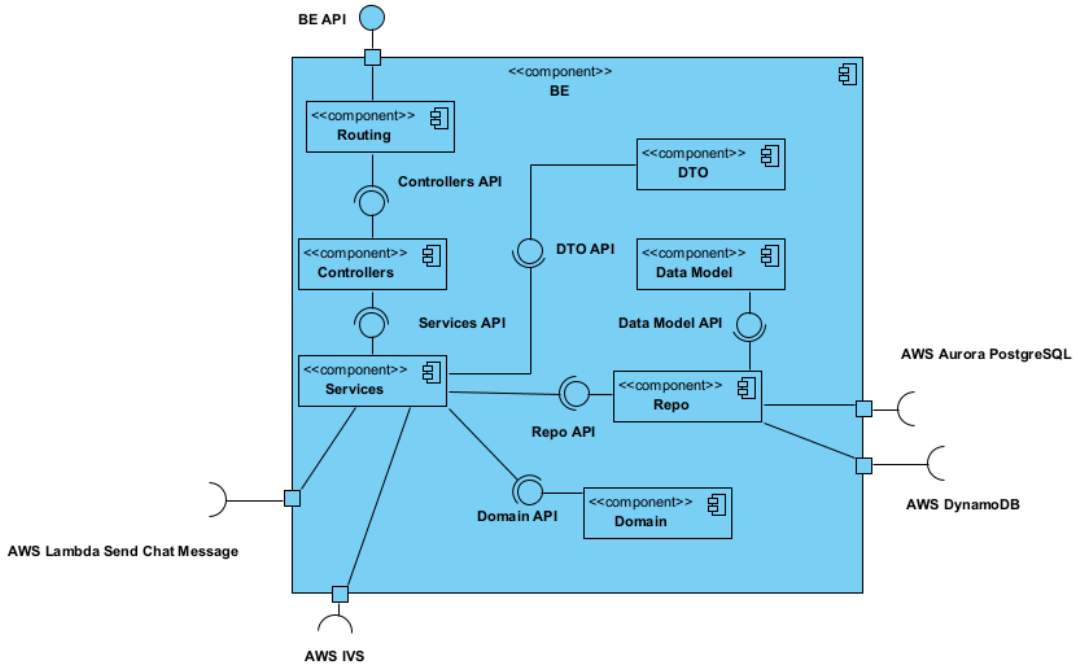

Figura 3.4: Vista Lógica do *Back-End* - Nível 3

Pela Figura 3.4 conseguimos observar as boas práticas do desenvolvimento de software, especificamente a separação de responsabilidades.

Os componentes desta vista são:

- **Routing** - Definem os *endpoints* da aplicação que o *front-ent* pode usar, e redireciona para os *Controllers*.
- **Controllers** - Estes recebem os pedidos e redirecionam para os diversos serviços da aplicação.
- **Services** - Contém a lógica da aplicação, servem de ponte para os repositórios.
- **Data Transfer Object (DTO)** - Este componente define o que tranferir de cada objeto pela aplicação.
- **Data Model** - Modelo de ligação ás bases de dados.
- **Repo** - Responsável pela persistência dos dados.
- **Domain** - Núcleo da aplicação, trazem o negócio para o código.

Também é possível perceber que o *back-end* integra na totalidade com as bases de dados e com os serviços que precisam de mais segurança como o IVS e a função de enviar mensagem pelo *chat*

Neste nível, também é apresentado a vista de implementação:

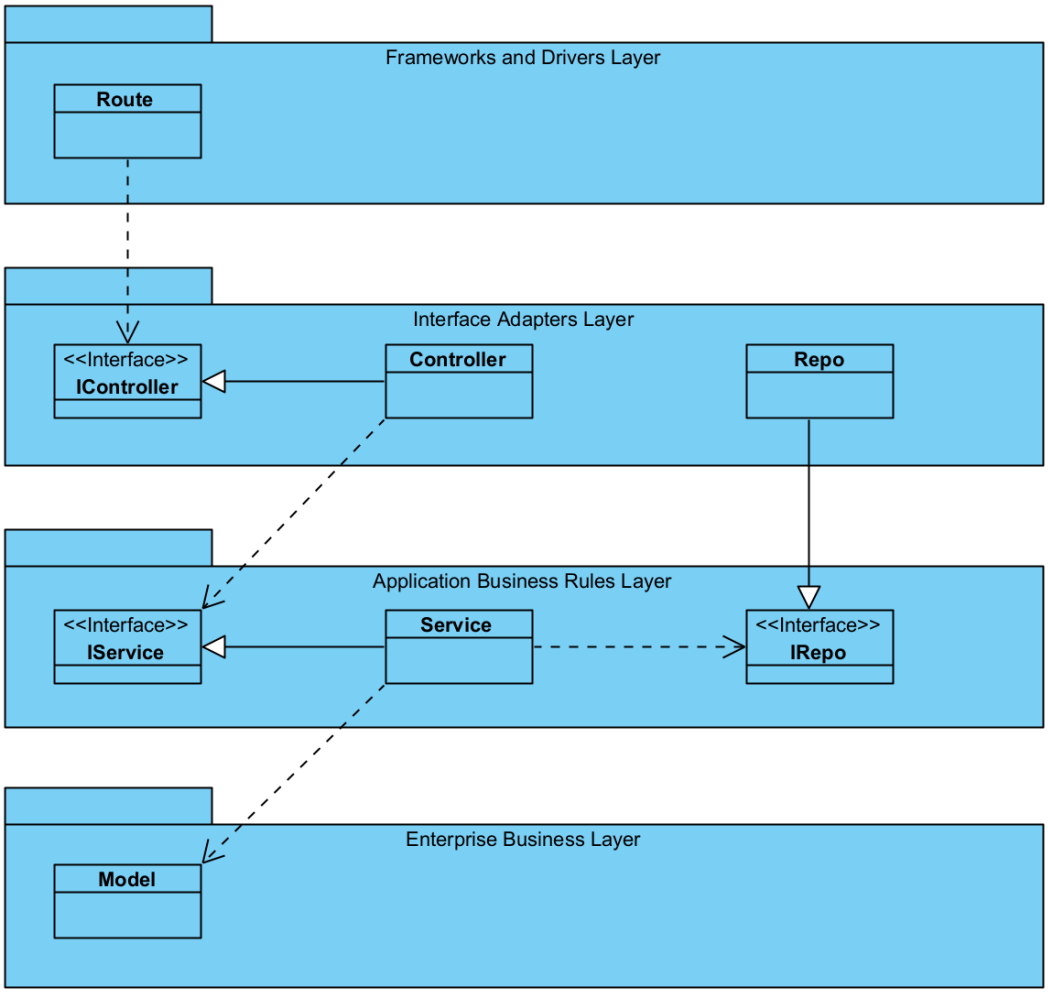

Figura 3.5: Vista de implementação

Nesta vista evidenciam-se camadas com dependências orientadas do exterior para o interior que terminam com interfaces. Esta em conformidade com o Dependency Inversion Principle (DIP). Assim, camadas internas não referenciam diretamente implementações externas, por exemplo: `Service` depende de `IRepo` e não de `Repo`.

A vista seguinte incide sobre o componente *front-end* da solução.

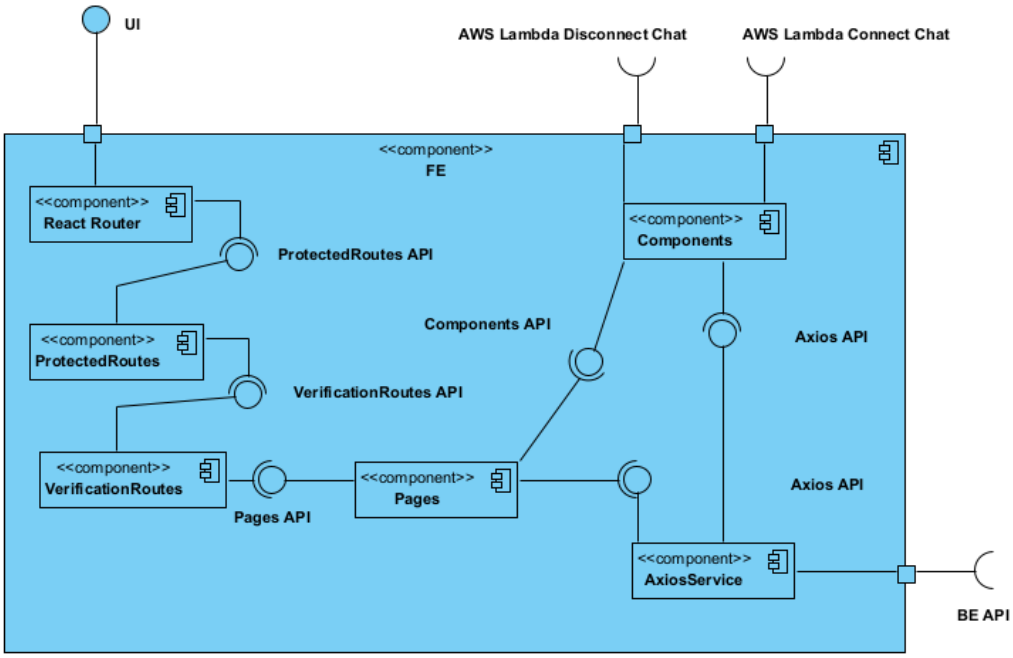

Figura 3.6: Vista Lógica Nível 3 - *Front-End*

A Figura 3.6 apresenta os seguintes componentes:

- **React Router** - Gere o sistema de navegação da aplicação
- **Protected Route** - verifica as permissões de acesso, no caso da solução verifica se o utilizador está autenticado.
- **Verification Route** - Aumenta a barreira de segurança, pois verifica se o utilizador tem permissão para assistir ou criar uma transmissão.
- **Pages** - Conjunto das páginas da aplicação.
- **Components** - Biblioteca de componentes reutilizáveis pelas páginas ou por outros componentes.
- **Axios Service** - Responsável pela comunicação com o *back-end*

3.4.5 Vista Física

Como toda a infraestrutura da solução reside na Amazon Web Services (AWS), nesta fase de desenho foi necessário aplicar os conhecimentos obtidos na formação.

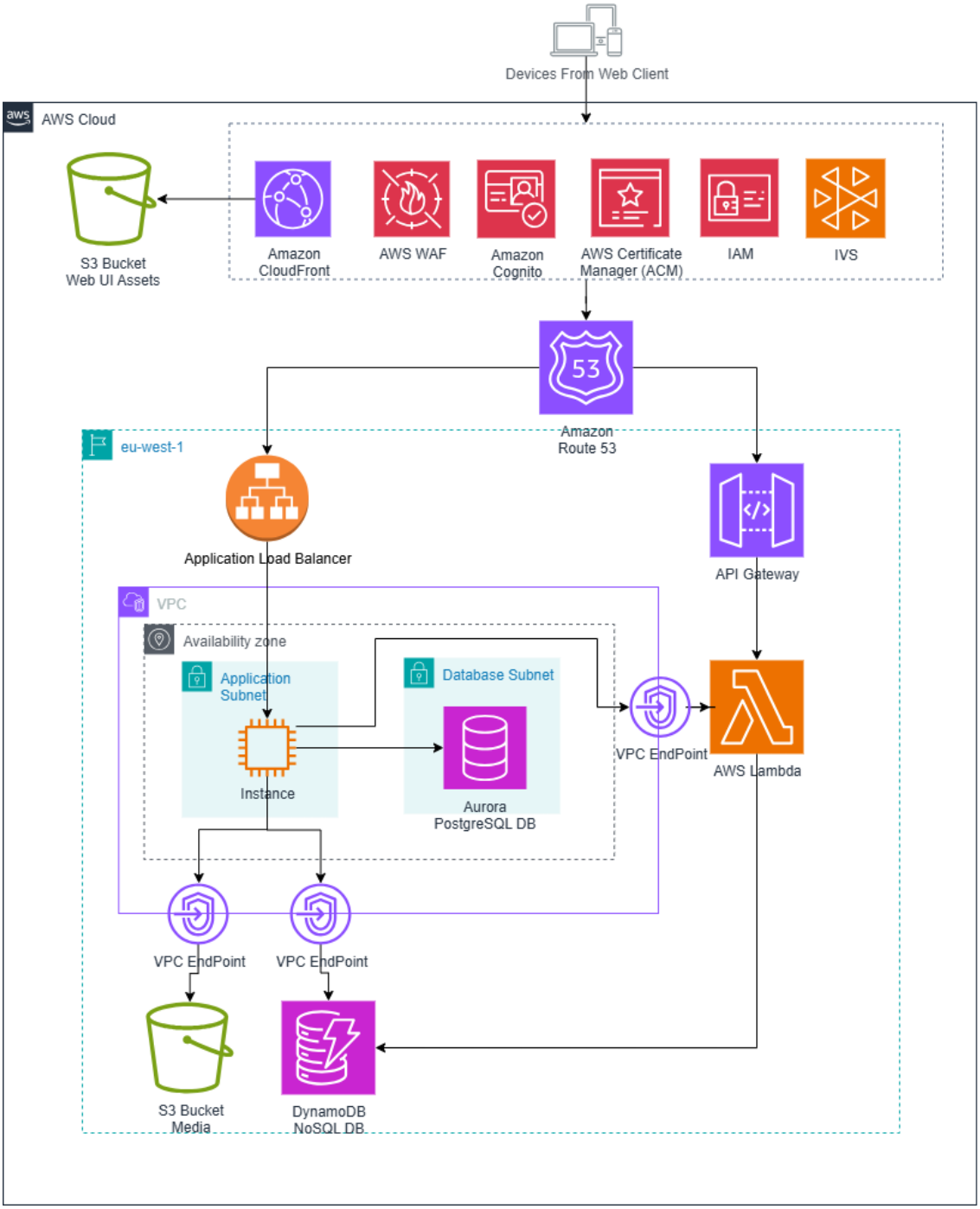

Figura 3.7: Vista física - Infraestrutura AWS

Na Figura 3.7 são apresentados os seguintes serviços:

Tabela 3.6: Serviços AWS e respetivo papel na solução

Serviço	**Descrição**
Amazon Route 53	Resolução de nomes do domínio da aplicação
Amazon CloudFront	Distribuição de conteúdos estáticos do S3
Amazon S3 (UI)	Contêm os conteúdos estáticos do *front-end*
Amazon S3 (Media)	Repositório de conteúdos multimédia (vídeos/fotos) .
AWS Web Application Firewall (WAF)	Controlo do tráfego na aplicação
AWS Certificate Manager (ACM)	Emite certificados Secure Sockets Layer (SSL) para a aplicação.
Application Load Balancer (ALB)	Distribui o tráfego de pedidos para diversas instâncias da aplicação.
Amazon Virtual Private Cloud (VPC)	Isola a rede da aplicação com, *subnets*, *security groups* e *routing table*.
VPC Endpoints	Permite aceder a serviços fora da VPC.
Amazon API Gateway	Expõem *enpoints* para o acesso as funções Lambda.
AWS Lambda	Função *serverless* que consegue fazer *build* de codigo independente.
Amazon Aurora PostgreSQL	Base de dados SQL
Amazon DynamoDB	Base de dados NoSQL
AWS IVS	Serviço gerido pela AWS que cria os canais de transmissão e as credenciais únicas de cada criador.
AWS Identity and Access Management (IAM)	Controla o acesso da aplicação.

Passando para uma explicação mais estruturada, em primeira instância, os dispositivos dos utilizadores acedem à aplicação, cujo nome é resolvido no **Amazon Route 53**.

O conteúdo do *front-ent* é alojados pelo **Amazon CloudFront** a partir de **Amazon S3**, com proteção **AWS WAF** e para segurança através do protocolo Transport Layer Security (TLS) via **Amazon Certificate Manager**.

As chamadas do *front-end* seguem dois percursos:

- Através de um **Application Load Balancer** para a APIREST do *back-end*.
- Através *API Gateway* para funções *AWS Lambda* quando adequado.

A persistência é feita por **Amazon Aurora PostgreSQL** (base de dados SQL), **DynamoDB** (base de dados NoSQL) e **S3** (para objetos como imagens e vídeos).

O **AWS IVS** suporta as transmissões em direto e o **IAM** controla o acesso aos recursos da infraestrutura.

Com estes serviços a solução garante escalabilidade, disponibilidade e segurança.

3.4.6 Modelo de Dados

Após a descrição do sistema, o modelo de dados foi desenhado para ajudar na compreensão do domínio e na construção das bases de dados.

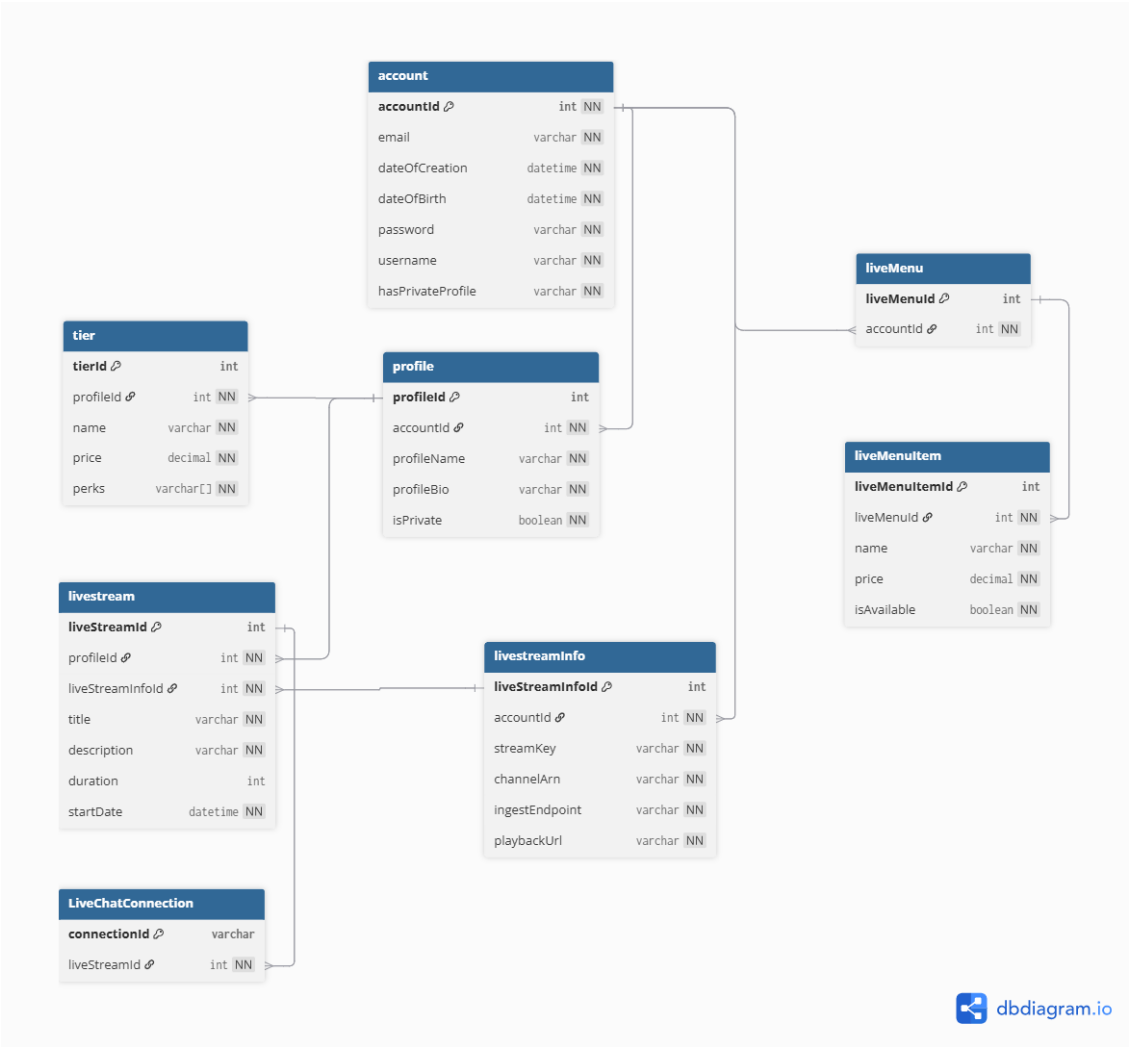

Figura 3.8: Modelo de Dados

Na Figura 3.8, está representado o modelo de dados adotado para a criação do dominio do código.

Adotou-se um modelo híbrido, que combina a base de dados relacional (PostgreSQL) e a base de dados não-relacional (DynamoDB).

A camada relacional preserva a integridade do modelo, onde são garantidas as regras de negocio e transações Atomicity, Consistency, Isolation, Durability (ACID).

A camada não-relacional serve para objetos de grande leitura e editáveis.

SQL

- **Account**: accountId, email, dateOfCreation, dateOfBirth, password, username, hasPrivateProfile.
- **Profile**: profileId, accountId, isPrivate.
- **Tier**: tierId, profileId.

- **LiveStreamInfo**: liveStreamInfoId, accountId, streamKey, channelArn, ingestEndpoint, playbackUrl.
- **LiveStream**: liveStreamId, profileId, liveStreamInfoId.
- **LiveMenu**: accountId, liveMenuId.
- **LiveMenuItem**: liveMenuItemId, liveMenuId, isAvailable.

DynamoDB

- **Profile**: profileId, name, bio.
- **Tier**: tierId, name, price, perks.
- **LiveStream**: liveStreamid, title, description, duration, startDate.
- **LiveMenuItem**: liveMenuItemId, name, price.
- **LiveChatConnection** : connectionId, liveStreamId.

Como as transmissões não serão gravadas, não houve a necessidade de fazer um modelo de dados para o *chat*.

3.4.7 Diagramas de sequência dos casos de uso

Nesta secção apresentam-se diagramas que explicam como cada caso de uso é executado ao nível das interações entre o ator e o sistema.

Para privilegiar a legibilidade e reduzir a poluição visual, os diagramas incluídos no corpo do texto encontram-se deliberadamente simplificados, representando apenas dois participantes: *Ator* e *Sistema*. O participante *Sistema* agrega, de forma abstrata, os componentes internos e respetivos fluxos.

As versões completas e detalhadas dos diagramas estão na secção dos anexos:

- Diagrama de Sequência do *back-end* A.6.
- Diagrama de Sequência do *front-end* A.5.
- Diagrama de Sequência do sistema A.4.

Casos de Uso C1 & C2

O diagrama de sequência 3.9, cobre os casos de uso que descrevem a configuração inicial do criador para criar um ambiente de transmissão. Descreve as validações necessárias para criar o título e a descrição, e inclui a persistência de dados.

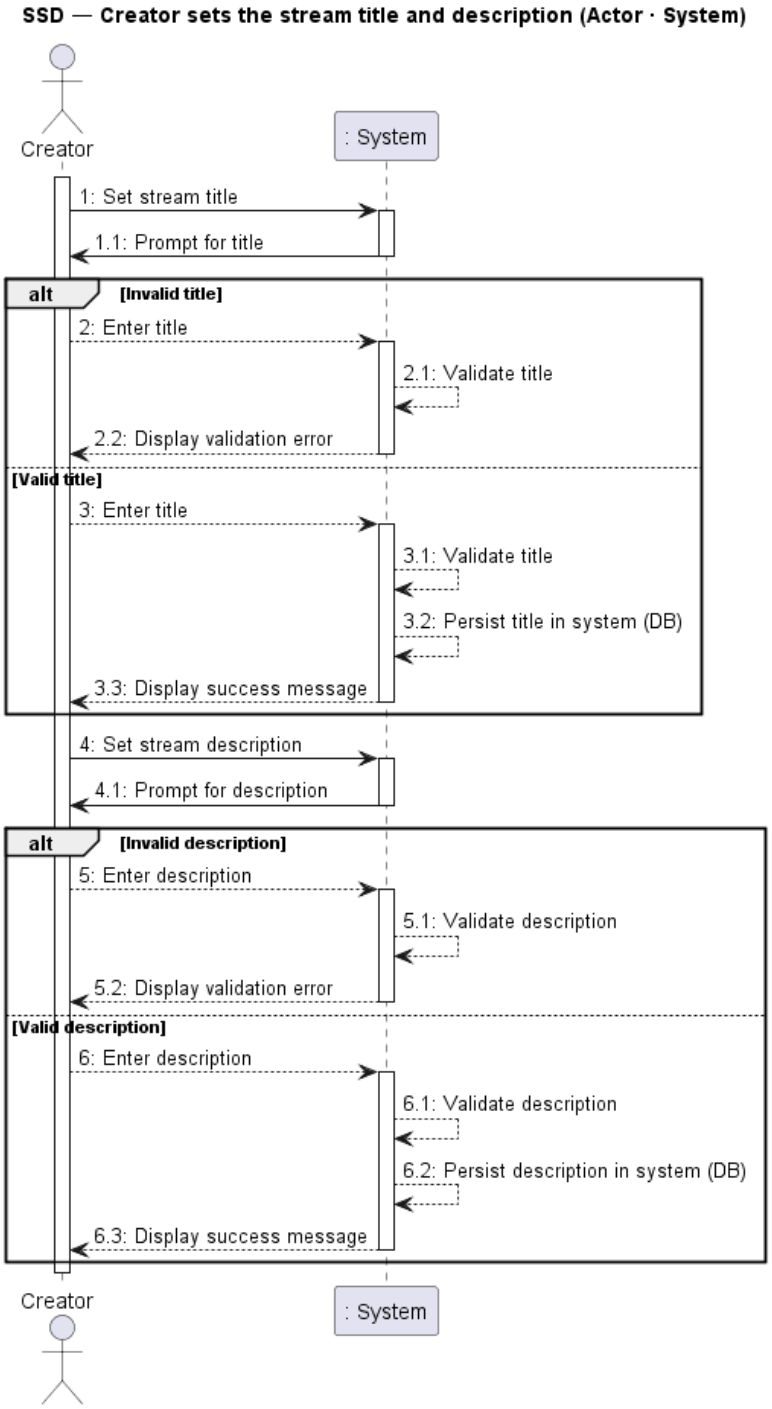

Figura 3.9: Diagrama de sequência do sistema para criar título e descrição da transmissão.

Casos de Uso C6 & C7 & C3

O diagrama de sequência 3.10, mostra a configuração da câmara e do microfone, através do *MediaStream* referido anteriormente no capítulo 2, com permissões pedidas pelo *browser*, ao utilizador e uma *preview* da transmissão.

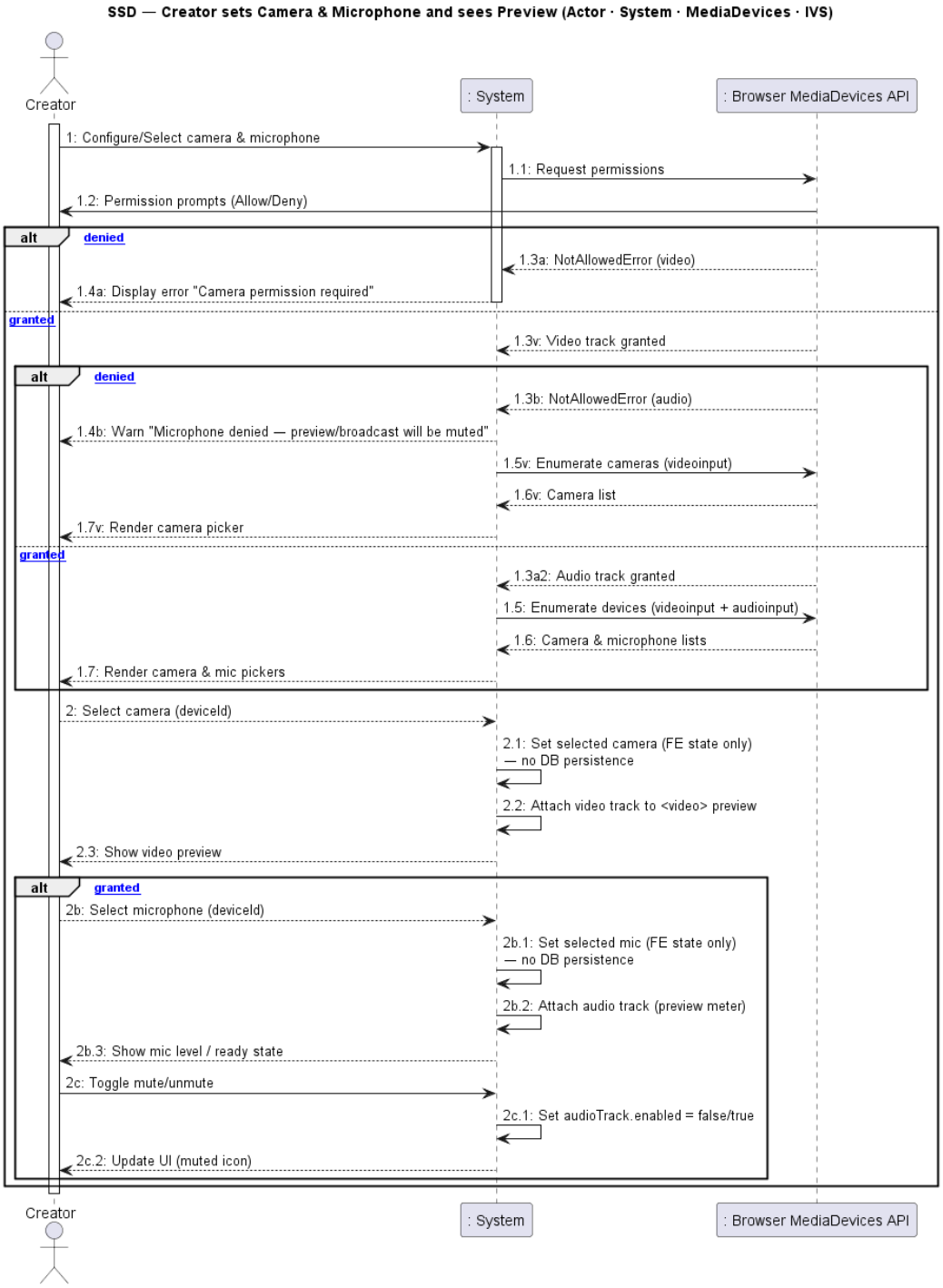

Figura 3.10: Diagrama de sequência do sistema para configurar câmara e microfone (ativação, permissões e teste).

Casos de Uso C8 & C9

O diagrama de sequência 3.11, descreve a ação de desativar e reativar o microfone e desligar a câmara. Ambas as ações geram notificações ao utilizador melhorando a sua experiência.

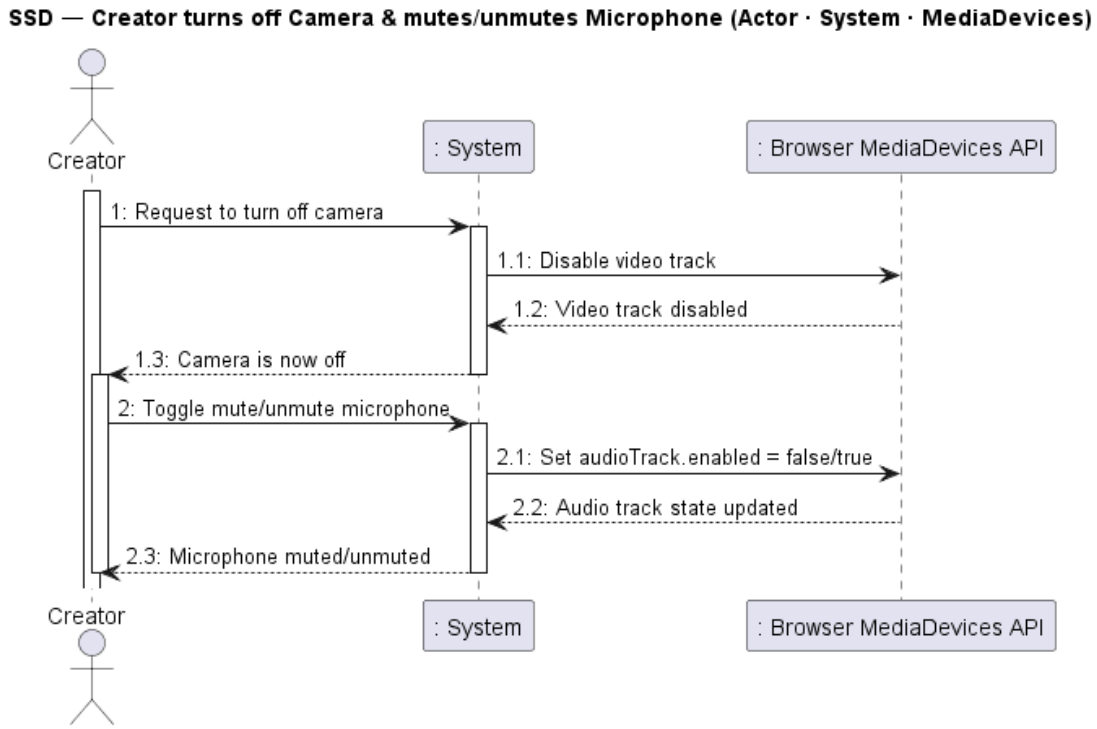

Figura 3.11: Diagrama de sequência do sistema para desativar/reativar câmara e microfone durante a transmissão.

Casos de Uso C5 & C10

O diagrama de sequência 3.12, simplifica e descreve a ação de começar uma transmissão, o sistema verifica todos os requisitos antes de criar a *stream*.

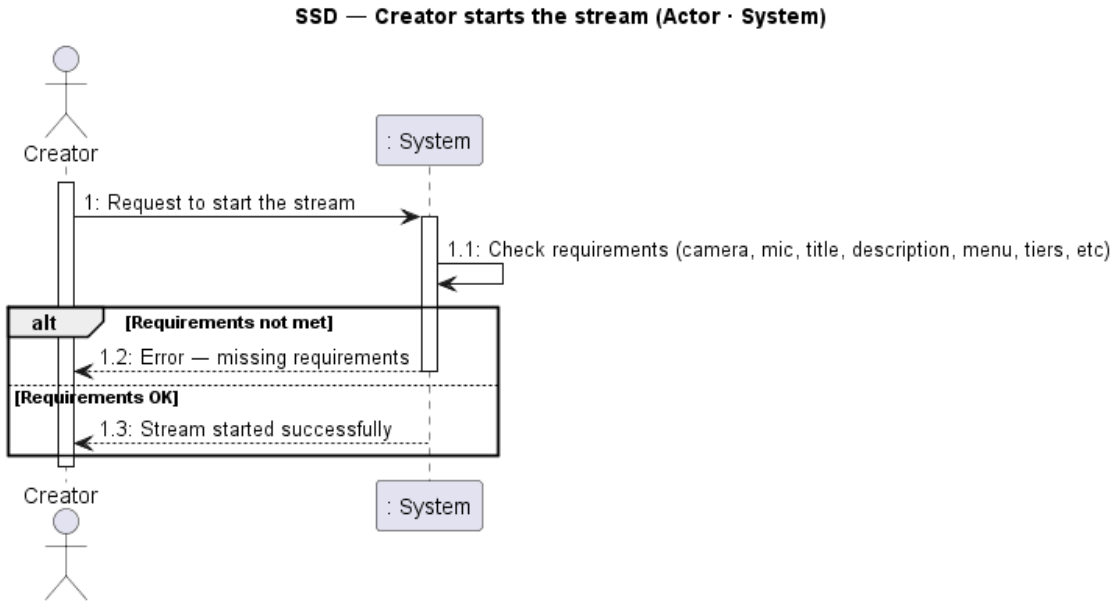

Figura 3.12: Diagrama de sequência do sistema para iniciar a transmissão (*start stream*).

O diagrama de sequência 3.13, descreve a simplicidade de acabar uma transmissão.

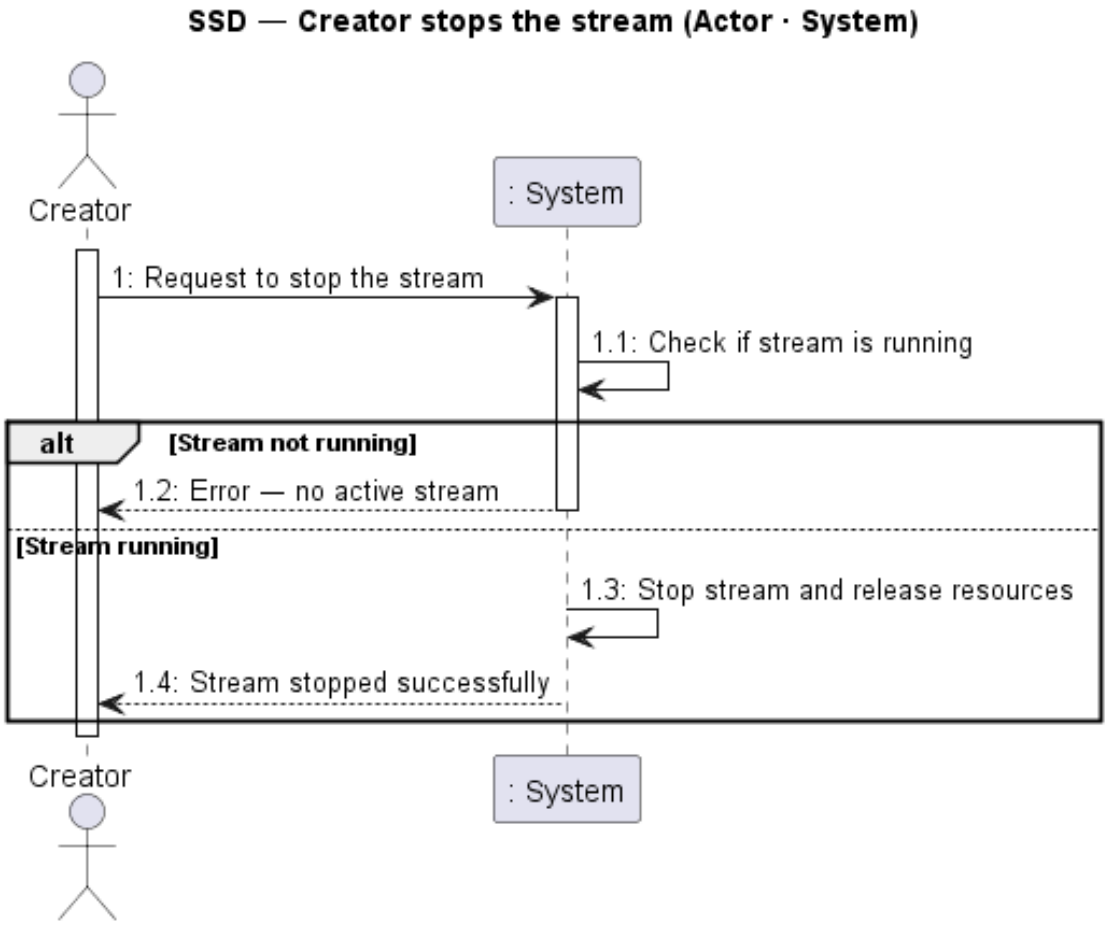

Figura 3.13: Diagrama de sequência do sistema para terminar a transmissão (*stop stream*).

Caso de Uso C11

O diagrama de sequência 3.14, descreve como o criador entra na sua própria página de *stream*, e por *web-socket*, liga-se a sala de *chat*. Após ligar-se fica capaz de receber mensagens em tempo real por outros utilizadores que estejam na mesma sala.

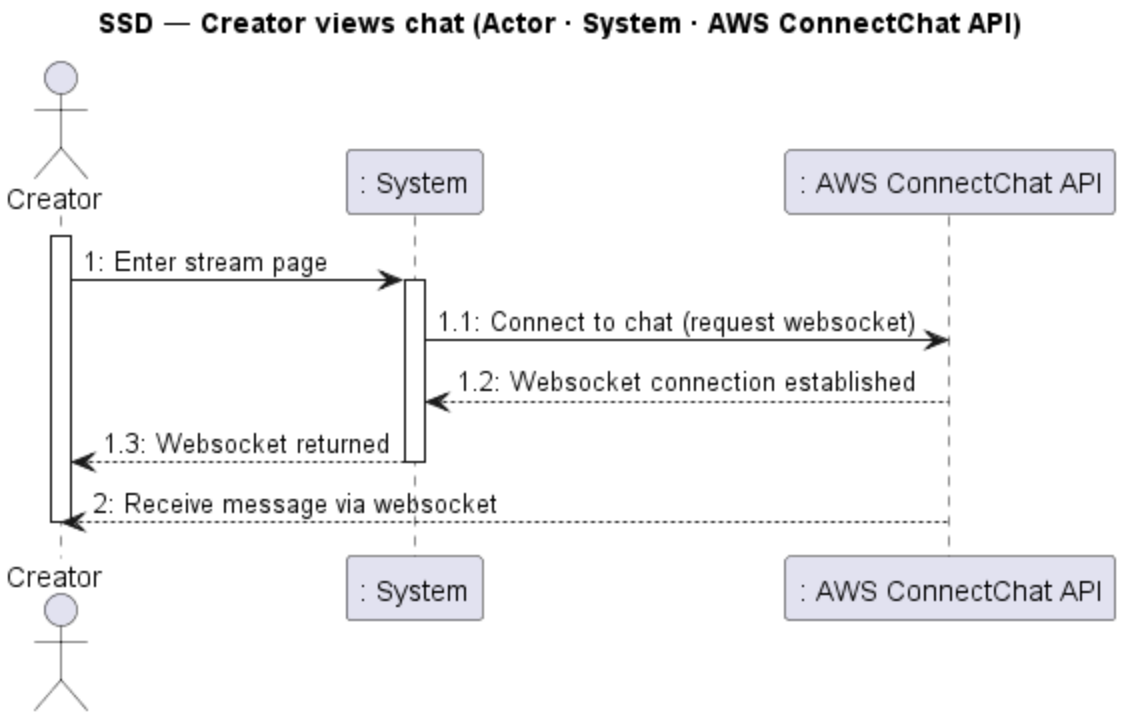

Figura 3.14: Diagrama de sequência do sistema para visualização do chat pelo criador.

Caso de Uso C12

O diagrama de sequência 3.15 especifica a interação ativa do criador com o chat: envio de mensagens, respostas, remoção e moderação. Detalham-se confirmações de entrega (*ack*), tratamento de *rate limits*, e feedback visual para estados em envio/erro. Prevê-se ainda a sincronização com serviços anti-spam e logs de auditoria.

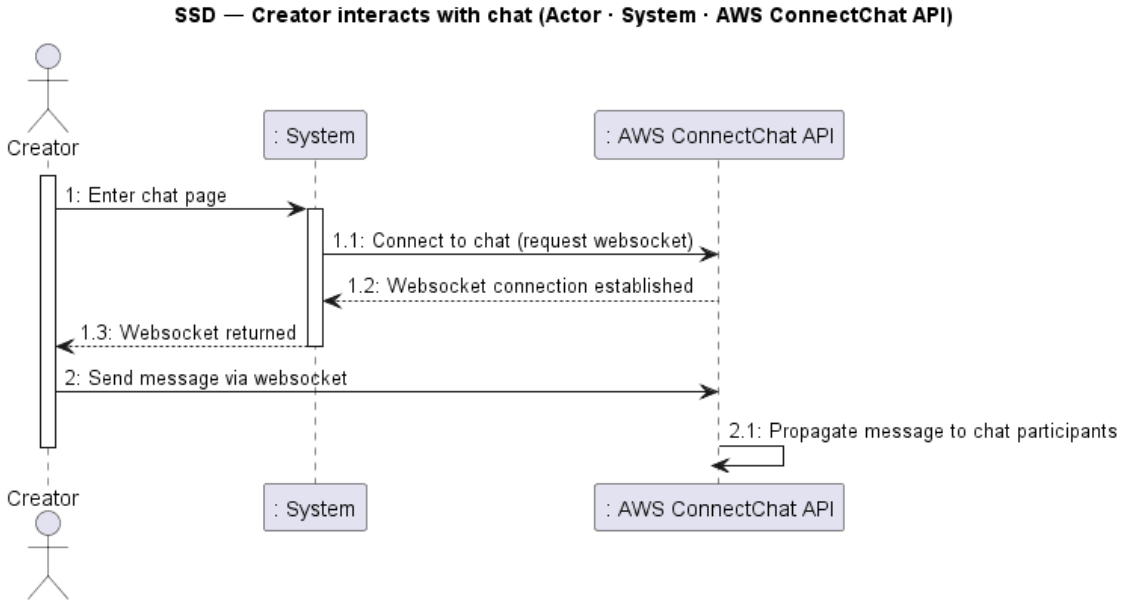

Figura 3.15: Diagrama de sequência do sistema para interação do criador no chat (envio e moderação).

Caso de Uso C13

O diagrama de sequência 3.16 foca a gestão e apresentação do *Live Menu* durante a transmissão. Descrevem-se a consulta do catálogo, a atualização de disponibilidade em tempo real e a reflexão imediata das alterações na interface. São também contemplados eventos de sincronização com o backend e métricas de uso.

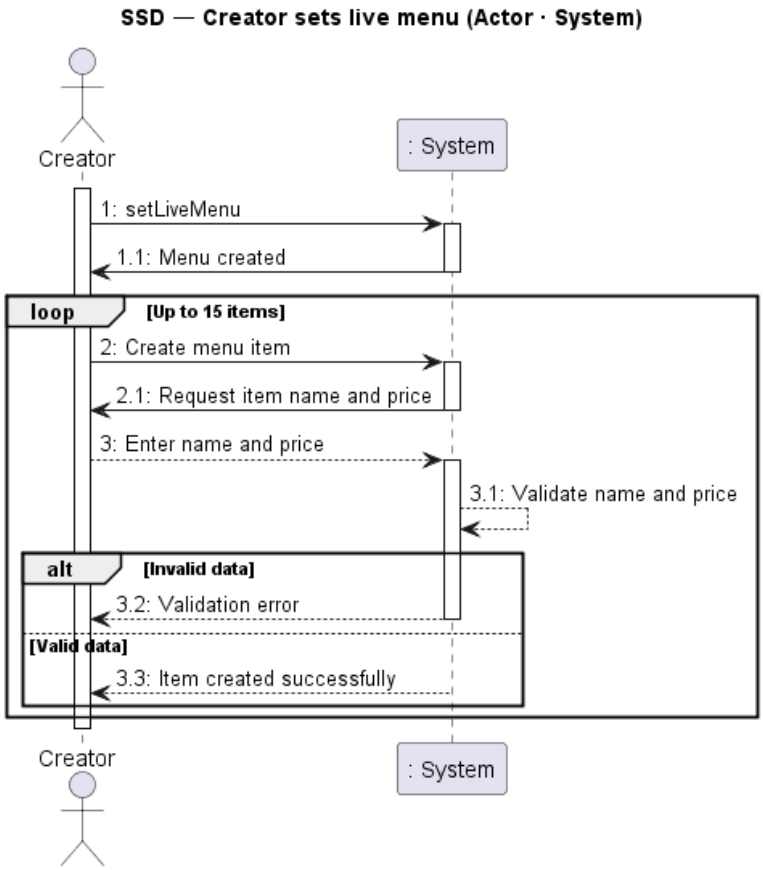

Figura 3.16: Diagrama de sequência do sistema para gestão e apresentação do *Live Menu*.

Caso de Uso C14

No diagrama de sequência 3.17, aborda-se a gestão de *tiers* de acesso e a sua seleção/atribuição à sessão.

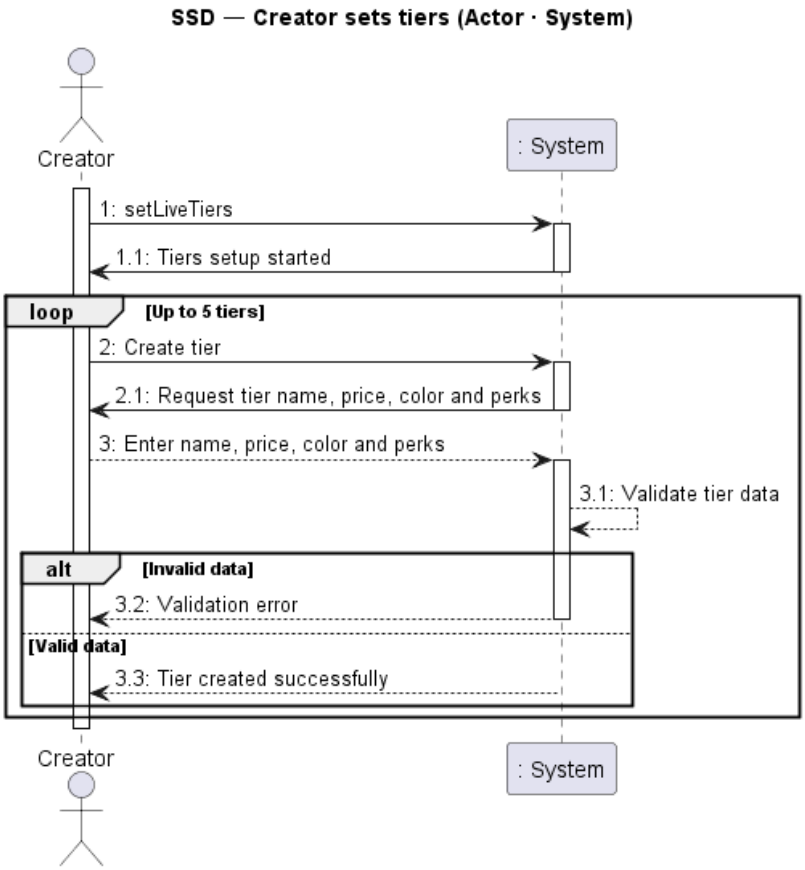

Figura 3.17: Diagrama de sequência do sistema para gestão e seleção de *tiers* de acesso.

Caso de Uso V1

O diagrama de sequência 3.18 descreve a experiência do *viewer* ao iniciar a visualização duma transmissão. Incluem-se a resolução de URL do canal, o estabelecimento da sessão de reprodução, a negociação de qualidade adaptativa e o tratamento de erros iniciais.

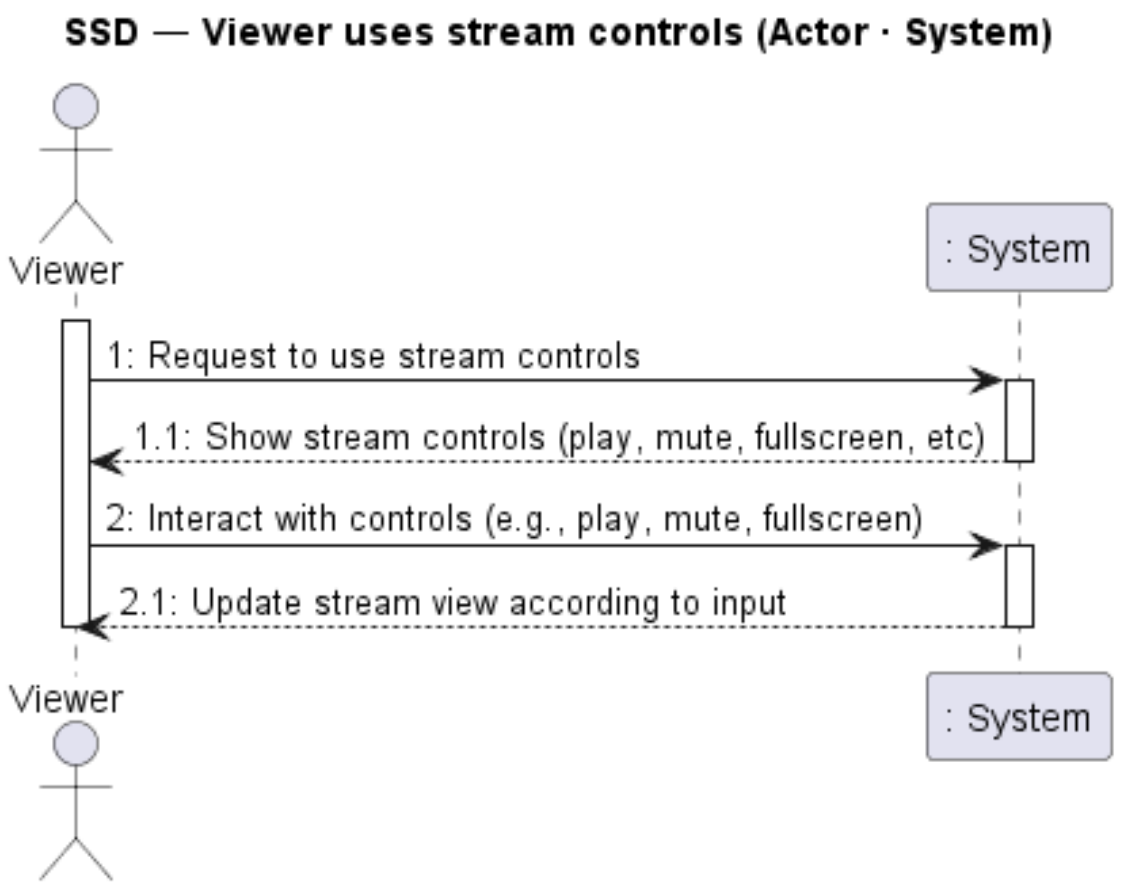

Figura 3.18: Diagrama de sequência do sistema para visualização da transmissão pelo utilizador.

Caso de Uso V2

O diagrama de sequência 3.19 aborda a interação do utilizador com o *chat* público/privado durante a transmissão. Explicita-se o envio e receção de mensagens em tempo real, limitações de frequência, e *feedback* de entrega. Prevê-se moderação automática e mecanismos de denúncia.

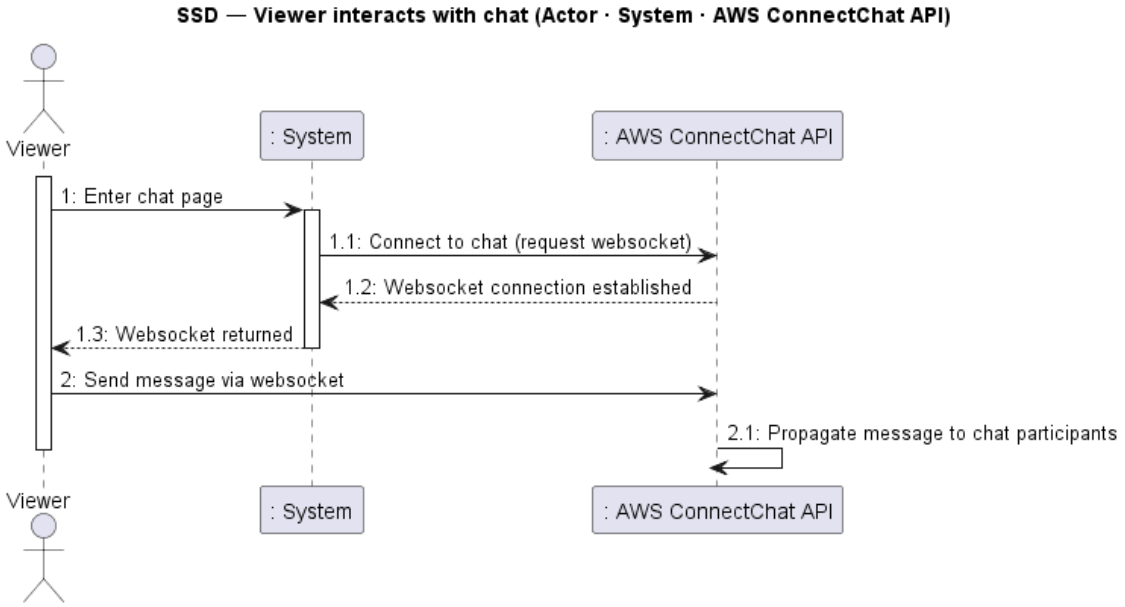

Figura 3.19: Diagrama de sequência do sistema para interação do utilizador no chat.

Caso de Uso V3

No diagrama de sequência 3.20 detalham-se os controlos de reprodução disponíveis para o utilizador: *play/pause*, volume, *mute*, ecrã inteiro e seleção de qualidade. Define-se a sincronização de estado entre UI e *player*, incluindo atalhos e acessibilidade. Consideram-se ainda indicadores de estado (buffering, erro) e mensagens de orientação.

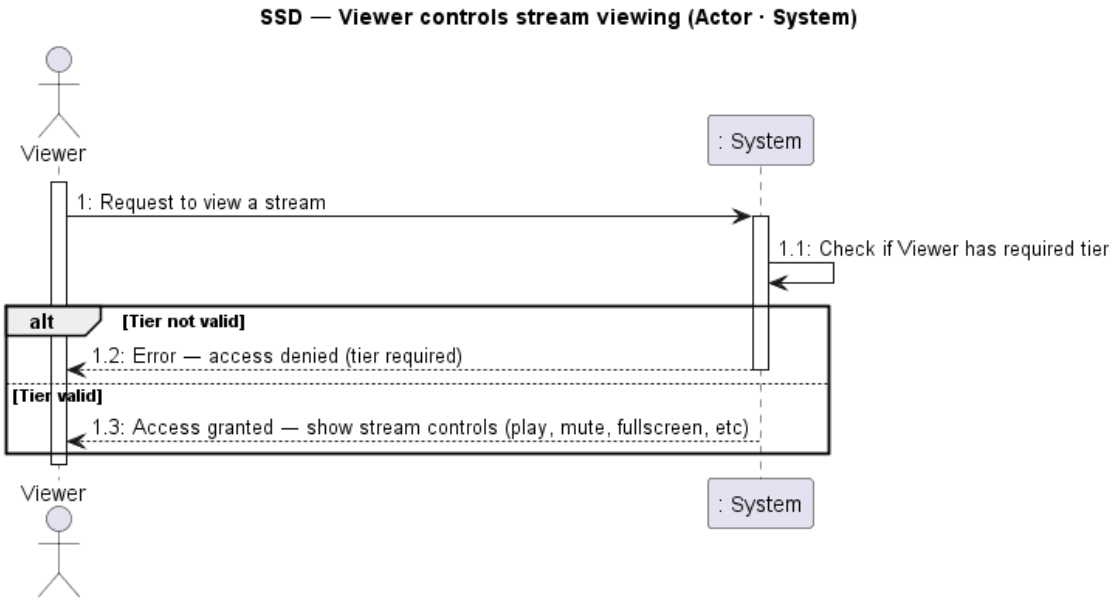

Figura 3.20: Diagrama de sequência do sistema para controlos de reprodução da transmissão pelo utilizador.

3.4.8 Alternativas ao design

Consoante o desenvolvimento, foram detetadas alternativas ao desenho da solução, nomeadamente na criação da transmissão. Nos caso de uso apresentados, as configurações como o título de a descrição são persistidos na base de dados antes de iniciar a *live*, por outro lado, poderia ser adequado só persistir nas bases de dado, apenas quando fosse começar uma transmissão.

Em alternativa ao modelo de como as salas de transmissões são feitas, seria prudente criar uma *web-socket*, para a sala em geral, pois para interações em tempo real, não seria necessário criar uma ligação nova a para cada funcionalidade. Exemplificando, quando um visualizador que doar ao criador durante a transmissão, precisa de ser despoletado uma ação de confirmação, e como esse serviço não está ligado a nenhuma "sala", não consegue ser em tempo real. Para resolver este caso foi desenvolvido um "scheduler"no *back-end*, que corre certos métodos em curtos espaços de tempo.

3.5 Conclusão

O capítulo apresentou uma visão completa e coerente do sistema, desde a caracterização do domínio e definição de requisitos até à arquitetura e modelos que os materializam. Clarificaram-se conceitos nucleares (*Account*, *Profile*, *Tier*, *LiveStream*) e respetivas relações, assegurando uma base semântica estável para o desenho técnico. Foram especificados requisitos funcionais (RF01–RF08) e não funcionais, enquadrados no modelo FURPS+, garantindo alinhamento entre necessidades de negócio, usabilidade, fiabilidade, desempenho e suportabilidade.

O desenho arquitetural, estruturado pelo modelo *C4*, evidenciou a separação *FE/BE*, responsabilidades por camadas e dependências internas, culminando numa vista física detalhada em *AWS*

O modelo de dados híbrido (relacional + NoSQL) foi alinhado com o domínio: integridade transacional em Aurora para entidades críticas e projeções e editabilidade em DynamoDB para leitura intensiva, reforçando tanto consistência como performance. Por fim, os diagramas de sequência consolidaram o comportamento por casos de uso.

Capítulo 4

Implementação da Solução

Neste capítulo, será apresentada a implementação da solução com recurso a alguns exemplos de desenvolvimento, interfaces de utilizador e a integração com serviços da AWS.

Poderá ser confirmada o uso da arquitetura modelada no capítulo 3, e de que forma os casos de uso e os requisitos funcionais foram implementados.

Nesta secção, são mostrados testes tanto de domínio, como testes *end-2-end*.

4.1 Descrição da implementação

Nesta secção, estão evidenciadas decisões, tecnologias e a forma como diferentes componentes da solução foram concretizados, assegurando a coerência com a arquitetura e a solução.

Para esta descrição, apresentou-se partes da solução e foram utilizados dados de teste, que não contemplam quaisquer informações reais.

4.1.1 Low Latency Chat

A solução descreve uma arquitetura em tempo real entre utilizadores ligados a uma mesma transmissão, garantindo o registo das ligações, difusão eficiente de mensagens e a eliminação dessa conecção após a saida da transmissão. Para tal efeito, foram usados alguns serviços AWS:

- API Gateway WebSocket
- AWS Lambda
- Amazon DynamoDB

A *web-socket* é composta por três rotas **$connect**, **$disconect** e **$sendChatMessage**, e cada uma dessas rotas quando alcançada pelo utilizador promove uma chamada às funções *serverless* (Lambda).

Segue-se o codigo das funções Lambda:

```
1     const { DynamoDBClient, PutItemCommand } = require("@aws-sdk/client-
      dynamodb");
2 const client = new DynamoDBClient({ region: "eu-west-1" });
3
4 exports.handler = async (event) => {
5   console.log("event", event);
6   const streamId = event.queryStringParameters?.streamId || "default";
```

```
7   console.log("streamId", streamId);
8   const connectionId = event.requestContext.connectionId;
9   console.log("connectionId", connectionId);
10
11  const command = new PutItemCommand({
12    TableName: "liveChat",
13    Item: {
14      streamId: { S: streamId },
15      connectionId: { S: connectionId },
16      timestamp: { N: `${Date.now()}` }
17    }
18  });
19
20  await client.send(command);
21  return { statusCode: 200 };
22 };
```

Listagem 4.1: Ligação ao Chat

Nesta função 4.1 o utilizador entra na página da transmissão que tem um *streamId* associado, e um *connectionId* fornecido pela API Gateway. Após a entrada o registo do utilizador é guardado na tabela *liveChat*.

```
1  const { DynamoDBClient, DeleteItemCommand, ScanCommand } = require("@aws
     -sdk/client-dynamodb");
2  const client = new DynamoDBClient({ region: "eu-west-1" });
3
4  exports.handler = async (event) => {
5    const connectionId = event.requestContext.connectionId;
6    const scan = new ScanCommand({
7      TableName: "liveChat",
8      FilterExpression: "connectionId = :conn",
9      ExpressionAttributeValues: {
10       ":conn": { S: connectionId }
11     }
12   });
13
14   const result = await client.send(scan);
15   const item = result.Items?.[0];
16   if (!item) return { statusCode: 200 };
17
18   const streamId = item.streamId.S;
19   console.log("streamId", streamId);
20   const del = new DeleteItemCommand({
21     TableName: "liveChat",
22     Key: {
23       streamId: { S: streamId },
24       connectionId: { S: connectionId }
25     }
26   });
27
28   await client.send(del);
29   return { statusCode: 200 };
30 };
```

Listagem 4.2: Remoção do Chat

Nesta função 4.2 a remoção do registo do utilizador é feita independentemente se ele sai da sala de transmissão diretamente, ou, se fecha o *browser*. Assim a base de dados não fica com registo errados.

```
exports.handler = async (event) => {
  let streamId, message;
  try {
    const body = typeof event.body === 'string' ? JSON.parse(event.body)
     : event.body;
    streamId = body.streamId;
    message = body.message;
  } catch (err) {
    console.error('Erro a fazer parse ao body:', event.body);
    return { statusCode: 400, body: 'Invalid JSON body' };
  }
  try {
    const connections = await ddb.query({
      TableName: 'liveChat',
      KeyConditionExpression: 'streamId = :s',
      ExpressionAttributeValues: {
        ':s': streamId
      },
    }).promise();

    // 2. Enviar a mensagem para cada connectionId
    const postCalls = connections.Items.map(async ({ connectionId }) =>
   {
      try {
        await apigw.postToConnection({
          ConnectionId: connectionId,
          Data: JSON.stringify({ message }),
        }).promise();
      } catch (err) {
        if (err.statusCode === 410) {
          console.log(`Stale connection: ${connectionId}`);
        } else {
          console.error(`Failed to send to ${connectionId}:`, err);
        }
      }
    });

    await Promise.all(postCalls);

    return { statusCode: 200, body: 'Message sent.' };
  } catch (err) {
    console.error('Error broadcasting message:', err);
    return { statusCode: 500, body: 'Internal error' };
  }
};
```

Listagem 4.3: Envio das Mensagens

Na função 4.3, dão entrada o *streamId* e a mensagem, é consultada a tabela *liveChat* onde são retirados os *id's* dos utilizadores presentes na transmissão, e procede-se ao envio das mensagens para cada ligação através da *web-socket*.

A imagem 4.1 mostram a interação de dois utilizadores:

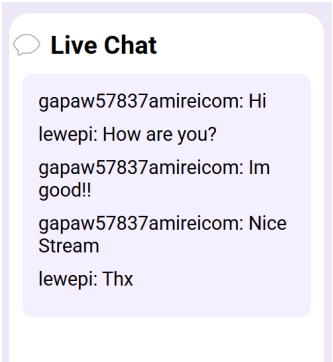

Figura 4.1: Componente do *low latency chat*

4.1.2 Tiers

Para o criador conseguir ter uma transmissão ao vivo, antes tem de definir níveis de acesso que permita algum utilizador entrar na sua página. Na figura 4.2

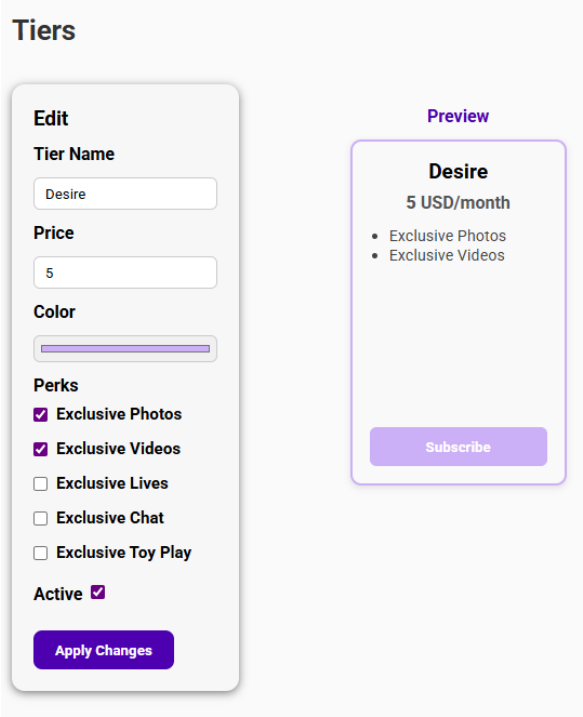

Figura 4.2: Edição e *Preview* de um escalão de acesso

Ao introduzir os dados necessários para criar um escalão: (*Tier Name*, *Tier Price*, *Color*, e *Perks*, o criador pode aplicar as mudanças/criar um escalão, e desde que tenha um nível com o benefício **Exclusive Lives**, consegue iniciar transmissão.

Do lado do utilizador normal, ao entrar no perfil privado do criador, depara-se com este menu 4.3:

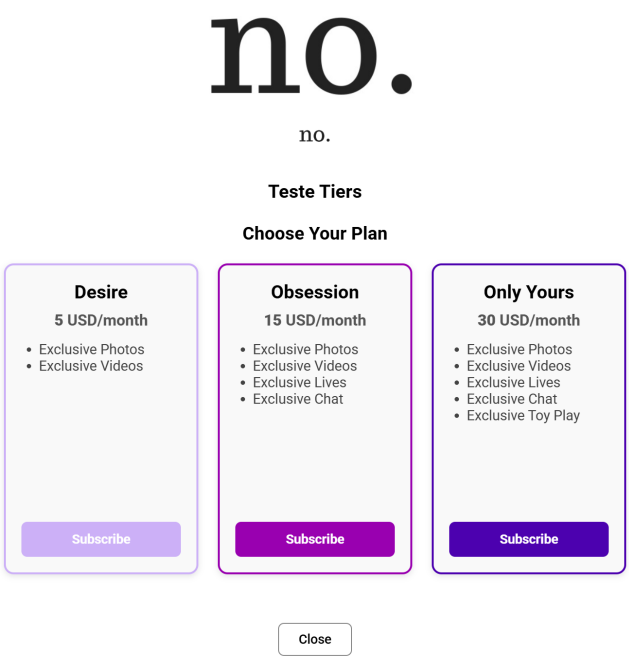

Figura 4.3: Área de seleção de um escalão

Nesta área o visiualizador, consegue ver a foto de perfil bem como a capa do criador de conteúdo.

4.1.3 Livestream Menu

Para tornar a transmissão mais mais interativa para o utilizador, o criador pode criar um menu com diversãs opções para fazer durante a transmissão. O menu é completamente editável, pode ter quinze opções com diferentes descrições e durante a transmissão o criador pode esconder essas opções.

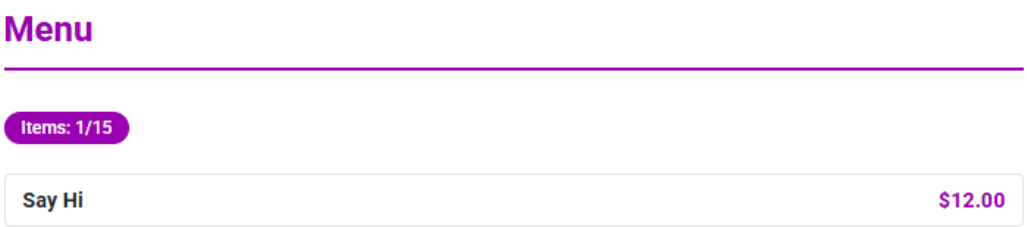

Figura 4.4: Menu de *Items* - Visualizador

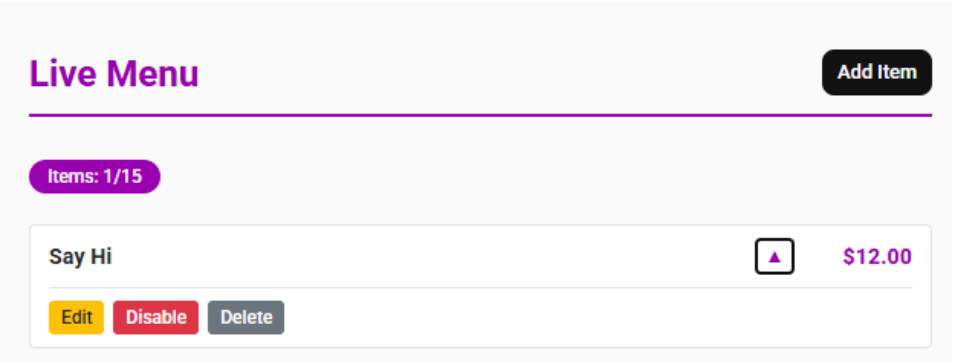

Figura 4.5: Menu de *Items* - Criador

As figuras 4.4 e 4.5 representam as duas interfaces na aplicação.

4.1.4 Livestream

A criação do canal de *streaming* é realizado pelo Software Development Kit (SDK) da AWS, esta operação devolve numa unica chamada, o **ARN do canal**, o **ingest endpoint** (servidor RTMP/RTMPS), o **playbackURL** (HLS para reprodução) e a **stream key**. 4.4

```
        private async createLiveStreamInfo(account: Account): Promise<
    Result<LiveStreamInfo>> {

        const client = new IvsClient({
            region: process.env.AWS_REGION || 'eu-west-1',
                credentials: {
                    accessKeyId: process.env.AWS_ACCESS_KEY_ID,
                    secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY,
                },
        });

        const input = {
            name: account.username.value,
            type: ChannelType.StandardChannelType,
            authorized: false
        };
        const command = new CreateChannelCommand(input);
        const response = await client.send(command);

        const liveStreamInfoOrError = LiveStreamInfo.create({
            accountId: account.accountId,
            channelArn: response.channel.arn,
            streamKey: response.streamKey.value,
            ingestEndpoint: response.channel.ingestEndpoint,
            playbackUrl: response.channel.playbackUrl,
        });

        return liveStreamInfoOrError;
    }
```

Listagem 4.4: Criação do Canal - IVS

Na Listagem 4.5 a entidade *LiveStreamInfo*, encapsula os artefactos retornados pelo AWS IVS com o *accountId* do criador. O *AggregateRoot*, valida a obrigatoriedade do *accountId* e garante que a aplicação só consome um único agregado.

```
interface LiveStreamInfoProps {
    channelArn: string;
    streamKey: string;
    ingestEndpoint: string;
    playbackUrl: string;
    accountId: AccountId;
}

export class LiveStreamInfo extends AggregateRoot<LiveStreamInfoProps> {
    get id(): UniqueEntityID {
        return this._id;
    }
    get livestreamInfoId(): LiveStreamInfoId {
        return LiveStreamInfoId.caller(this.id);
    }
    get channelArn(): string {
        return this.props.channelArn;
    }
    get streamKey(): string {
        return this.props.streamKey;
    }
    get ingestEndpoint(): string {
        return this.props.ingestEndpoint;
    }
    get playbackUrl(): string {
        return this.props.playbackUrl;
    }
    get accountId(): AccountId {
        return this.props.accountId;
    }
    set channelArn(channelArn: string) {
        this.props.channelArn = channelArn;
    }
    set streamKey(streamKey: string) {
        this.props.streamKey = streamKey;
    }

    set ingestEndpoint(ingestEndpoint: string) {
        this.props.ingestEndpoint = ingestEndpoint;
    }
    set playbackUrl(playbackUrl: string) {
        this.props.playbackUrl = playbackUrl;
    }

    private constructor(props: LiveStreamInfoProps, id?: UniqueEntityID)
    {
        super(props, id);
    }

    static create(props: LiveStreamInfoProps, id?: UniqueEntityID):
    Result<LiveStreamInfo> {
        const guarderProps= [
            { argument: props.accountId, argumentName: 'accountId' },
            { argument: props.channelArn, argumentName: 'channelArn' },
            { argument: props.streamKey, argumentName: 'streamKey' },
            { argument: props.ingestEndpoint, argumentName: '
    ingestEndpoint' },
            { argument: props.playbackUrl, argumentName: 'playbackUrl'
    },
```

```
        ];
        const guardResult = Guard.againstNullOrUndefinedBulk(
    guarderProps);

        if (!guardResult.succeeded) {
            return Result.fail<LiveStreamInfo>(guardResult.message);
        }
        return Result.ok<LiveStreamInfo>(new LiveStreamInfo(props, id));
    }

}
```

Listagem 4.5: Classe de dominio - LiveStreamInfo

A figura 4.6 apresenta a interface de criador durante uma transmissão, e a listagem 4.6, liberta os recursos antigos, caso o criador escolha mudar de câmara, cria um objeto para pedir ao navegador tanto vídeo como áudio e guarda tanto os dispositivos como a câmara como o microfone.

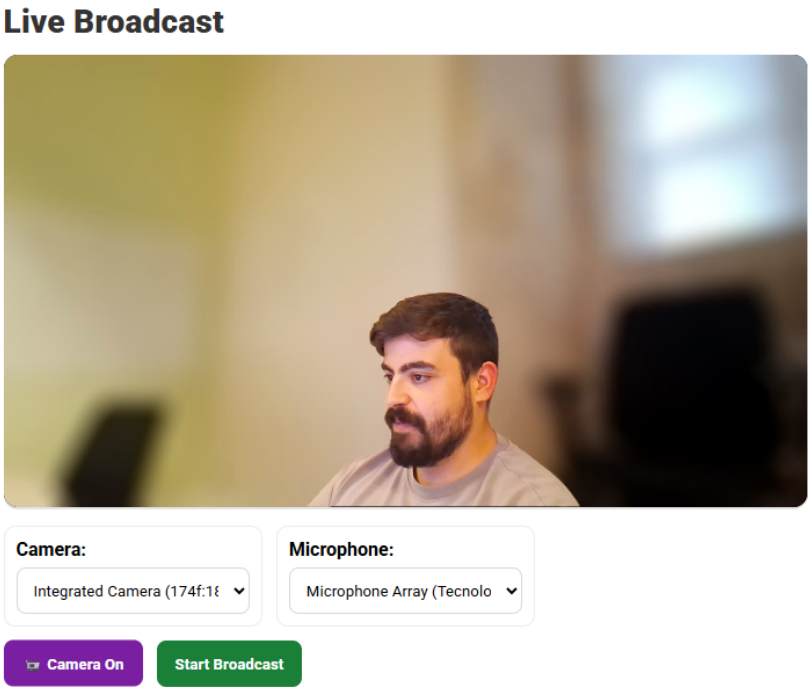

Figura 4.6: Interface de Criador

```
const getMediaStream = async () => {
    try {
      if (mediaStreamRef.current) {
        mediaStreamRef.current.getTracks().forEach(track => track.stop()
    );
        mediaStreamRef.current = null;
      }

      const constraints = {
        video: {
          deviceId: selectedVideoDevice ? { exact: selectedVideoDevice }
     : undefined,
```

```
11          width: { ideal: 1920 },
12          height: { ideal: 1080 }
13        },
14        audio: {
15          deviceId: selectedAudioDevice ? { exact: selectedAudioDevice }
    : undefined
16        }
17      };
18
19      const stream = await navigator.mediaDevices.getUserMedia(
    constraints);
20      mediaStreamRef.current = stream;
21      return stream;
22    } catch (err) {
23      setError("Error getting media stream: " + err.message);
24      return null;
25    }
26  };
```

Listagem 4.6: Retirar a *MediaStream* do navegador

A figura 4.7 apresenta a interface de visualizador durante uma transmissão, nela estão apresentados as configurações de utilizador como: *play*, *mute*, *fullscreen* e uma definição para voltar ao instante em direto na transmissão.

Para renderizar o a transmissão foi usado o protocolo HLS e a sua versão com menos latência (LL-HLS), o componente consome o *URL* de *playback* e recebe informação diretamente da AWS.

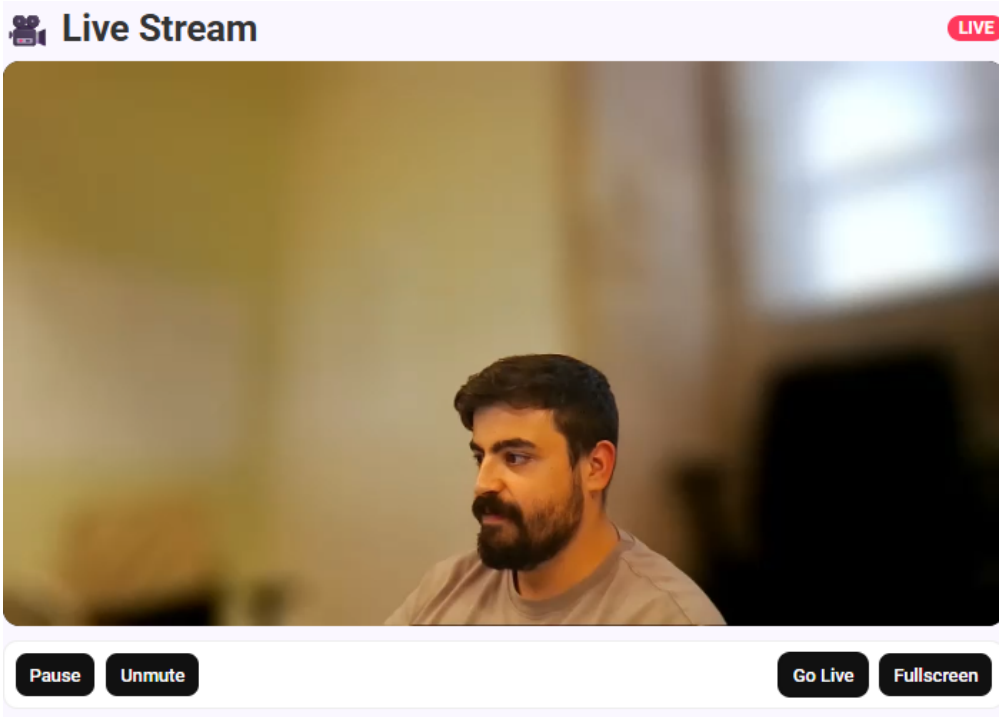

Figura 4.7: Interface de Utilizador

4.2 Testes

Nesta secção, são apresentadas evidências de testes tanto de domínio como *end-2-end*.

Testes de Domínio

- **LiveMenu**
  - Cria-se um *LiveMenu* válido.
  - Falha quando `accountId` está ausente.
- **LiveMenuItem**
  - Cria-se um *LiveMenuItem* válido.
  - Falha quando `name` está ausente.
  - Falha quando `price` está ausente.
  - Falha quando `liveMenuId` está ausente.
  - Falha quando `name` está vazio.
  - Falha quando `name` excede o comprimento permitido.
  - Falha quando `price` é negativo.
- **LiveStream**
  - Cria-se um *LiveStream* válido.
  - Falha quando `startTime` está ausente.
  - Falha quando `livestreamInfo` está ausente.
- **LiveStreamInfo**
  - Cria-se um *LiveStreamInfo* válido.
  - Falha quando `accountId` está ausente.
  - Falha quando `channelArn` está ausente.
  - Falha quando `streamKey` está ausente.
  - Falha quando `ingestEndpoint` está ausente.
  - Falha quando `playbackUrl` está ausente.
- **Tier**
  - Cria-se um *Tier* válido.
  - Falha quando `perks` está vazio.
  - Falha quando `name` está ausente.
  - Falha quando `colorCode` está ausente.
  - Falha quando `price` está ausente.

Na Figura 4.9, são apresentados os resultados dos testes.

```
LiveMenu Domain
  ✓ should create a valid LiveMenu
  ✓ should fail if accountId is missing

LiveMenuItem Domain
  ✓ should create a valid LiveMenuItem
  ✓ should fail if name is missing
  ✓ should fail if price is missing
  ✓ should fail if liveMenuId is missing
  ✓ should fail if name is empty
  ✓ should fail if name is too long
  ✓ should fail if price is negative

LiveStream Domain
  ✓ should create a valid LiveStream
  ✓ should fail if startTime is missing
  ✓ should fail if livestreamInfo is missing

LiveStreamInfo Domain
  ✓ should create a valid LiveStreamInfo
  ✓ should fail if accountId is missing
  ✓ should fail if channelArn is missing
  ✓ should fail if streamKey is missing
  ✓ should fail if ingestEndpoint is missing
  ✓ should fail if playbackUrl is missing

Tier Domain
  ✓ should create a valid Tier
  ✓ should fail if perks is empty
  ✓ should fail if name is missing
  ✓ should fail if colorCode is missing
  ✓ should fail if price is missing
```

Figura 4.8: Testes de Domínio

Para realizar os testes de domínio usaram-se algumas bibliotecas do **NPM** como **Jest**, **Mocha** e **Chai**.

```
1   it("should create a valid LiveStreamInfo", () => {
2       const props = {
3           accountId,
4           channelArn: "arn:aws:ivs:channel",
5           streamKey: "key123",
6           ingestEndpoint: "endpoint",
7           playbackUrl: "url"
8       };
9       const result = LiveStreamInfo.create(props);
10      expect(result.isSuccess).to.equal(true);
11      expect(result.getValue().accountId).to.equal(accountId);
12      expect(result.getValue().channelArn).to.equal("arn:aws:ivs:
    channel");
13      expect(result.getValue().streamKey).to.equal("key123");
14      expect(result.getValue().ingestEndpoint).to.equal("endpoint");
15      expect(result.getValue().playbackUrl).to.equal("url");
16  });
```

Listagem 4.7: Teste de criação do LiveStreamInfo

Neste teste, os *props* são construidos com todos os atributos necessários. De seguida, invoca-se o método *create* da classe *LiveStreamInfo*, que valida se os atributos foram bem definidos. Por fim, o teste passa de o método **create** devolver uma entidade igual ao respetivo valor fornecido pelos *props*.

Testes end-2-end

O principal objetivo destes testes é garantir que uma transmissão pode ser iniciada e parada de forma consistente, incluindo ciclos rápidos e proteção contra dois cliques. Também é importante capturar os estados da interface de utilizador, e manter moderar o *low latency chat*.

Livestream Tests

- Começar e parar a transmissão.
- Vários ciclos de começo e termino de transmissão.
- Prevenir cliques duplos.

Low Latency Chat Tests

- Enviar mensagem normal.
- Tamanho mínimo da mensagem.
- Tamanho máximo da mensagem.
- Envio de *emojis*.
- Validação da pontuação (acentos).
- Validação de caracteres especiais.

Os seguintes excertos de código demonstram a configuração do ambiente *end-2-end*. A lista 4.8 parametriza o contexto do *browser* com as permissões e os dispositivos. A lista 4.9 e a lista 4.10 organizam o ambiente de testes, realizam a autenticação, navegação para a página de transmissão, as operações de iniciar e parar a transmissão. Por fim, a lista 4.11 mostra o código literal do teste.

```
1
2  test.use({
3   browserName: 'chromium',
4   channel: 'chrome',
5   permissions: ['camera', 'microphone'],
6   launchOptions: {
7     args: [
8       '--use-fake-ui-for-media-stream',
9       '--use-fake-device-for-media-stream',
10      '--no-sandbox',
11    ],
12  },
13 });
```

Listagem 4.8: Criação do ambiente de testes

```
1
2 test.describe('Private Stream Tests (refactored)', () => {
3   test.beforeEach(async ({ page }) => {
4     const auth = new Auth(page);
5     const profile = new Profile(page);
6
7     await auth.openHome('https://dev.onlyyestest.com/');
8     await auth.openSignIn();
9     await auth.fillEmailAndPassword(email, password);
10    await auth.submitSignIn();
```

```
11
12      await profile.openSettings();
13      const stream = new Stream(page);
14      await stream.openPrivateLive();
15    });
```

Listagem 4.9: Configuração do ambiente de testes

```
1
2  class Stream {
3    private readonly btnStart;
4    private readonly btnStop;
5
6    constructor(private readonly page: Page) {
7      this.btnStart = page.getByRole('button', { name: 'Start Broadcast'
     });
8      this.btnStop  = page.getByRole('button', { name: 'Stop Broadcast' })
     ;
9    }
10
11   async start(): Promise<void> {
12     await this.expectStartVisible();
13     await this.btnStart.click();
14     await this.expectStopVisible();
15   }
16
17   async stop(): Promise<void> {
18     await this.expectStopVisible();
19     await this.btnStop.click();
20     await this.expectStartVisible();
21   }
22
23   async expectStartVisible(): Promise<void> {
24     await expect(this.btnStart).toBeVisible({ timeout: 8000 });
25   }
26   async expectStopVisible(): Promise<void> {
27     await expect(this.btnStop).toBeVisible({ timeout: 5000 });
28   }
29 }
```

Listagem 4.10: Operações de inicio e fim de transmissão

```
1
2    test('Start and stop broadcast stream', async ({ page }) => {
3      const stream = new Stream(page);
4      await stream.start();
5      await stream.stop();
6    });
7  });
```

Listagem 4.11: Teste de inicio e fim de transmissão

4.3 Avaliação da solução

Na empresa, foram testados alguns *endpoints* para verificar o comportamento da solução na infraestrutura AWS. Na imagem **??**, mostra um teste de carga realizado num dos destes

endpoints. Conseguimos observar que em média, a resposta dos pedidos é 200 milissegundos, o que para a empresa foi satisfatório.

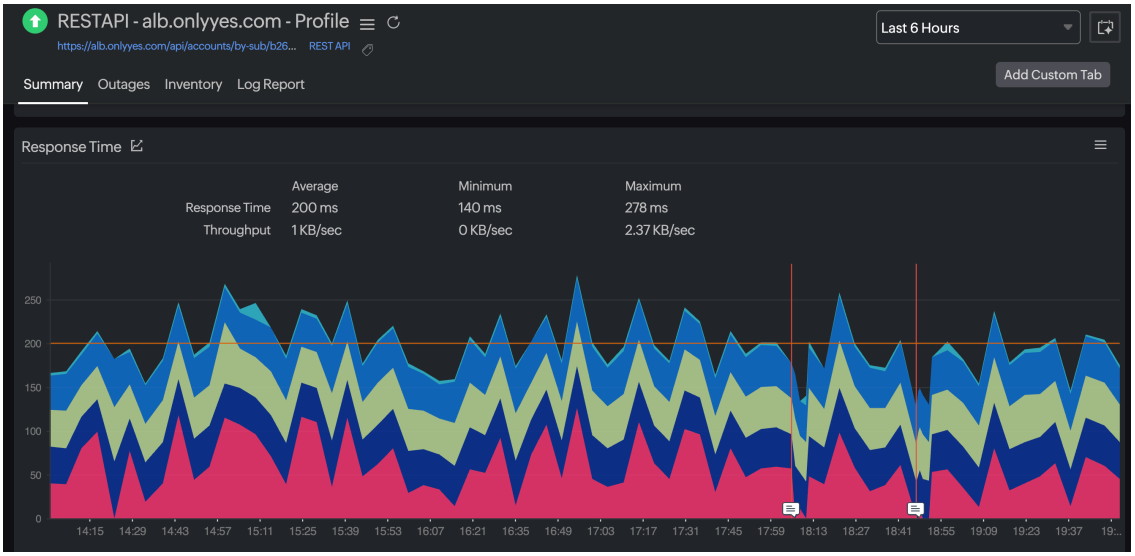

Figura 4.9: Teste de Carga

Capítulo 5

Conclusões

O capítulo final, sintetiza as principais conclusões que se obteve neste projeto. Serão analisados os objetivos do capítulo 1.

5.1 Objetivos concretizados

Na Tabela 5.1 encontram-se os objetivos e o grau de conclusão dos mesmos.

Tabela 5.1: Grau de Conclusão dos Objetivos

Objetivo	**Conclusão**
Modelação e criação de escalões de acesso para controlar o acesso à transmissão	Concluído
Criação de canal único por criador	Concluído
Desenvolvimento da interface do criador	Maioritariamente Concluído
Desenvolvimento da interface do utilizador	Maioritariamente Concluído
Garantia de privacidade do Criador	Concluído
Implementação do *chat* em direto	Concluído

5.2 Limitações e trabalho futuro

Mesmo que a maior parte dos objetivos tenham sido concretizados, persistem limitações na experiência "ao vivo" relacionadas com latência e consistência de reprodução. A solução baseada em HLS de baixa latência é adequada para difusão massiva, mas não para interações sub-segundo. Recomenda-se a normalização do uso do *IVS Player* e a reserva de *hls.js* apenas como *fallback* controlado. Propõe-se a criação de *dashboards* operacionais com métricas de *player* e rede do utilizador.

No *chat* em tempo real, a arquitetura com API Gateway WebSocket e DynamoDB cumpre os requisitos, mas pode ser otimizada. Sugere-se *heartbeats* periódicos para detetar ligações obsoletas e limitação de taxa por canal para mitigar *backpressure*. A moderação pode evoluir de regras básicas para muting, *rate limits* adaptativos e filtragem de *spam*.

5.3 Apreciação final

Em termos globais, considera-se que os objetivos principais do MVP são atingidos. O objetivo principal foi alcançado: realizar uma transmição em direto com poucos cliques. As

restantes funcionalidades como os *tiers*, o *chat* e o menu de opções operam com tempos de resposta adequados e fornecem uma base sólida para evolução.

O estágio permitiu ajustar e melhorar as boas práticas que desenvolvi ao longo do curso (DDD, CI/CD). Para além disso proporcionou uma experiencial de trabalho com a AWS, um serviço de computação na *cloud*. Os serviços da AWS estendem-se para diversos ramos, *data*, *DevOps*, cibersegurança...

Por fim, a participação neste projeto de uma *start-up*, trouxe momentos

Por fim, a convivência direta na construção de uma *start-up* proporcionou ganhos difíceis de reproduzir em contextos mais estáveis: proximidade real com a definição do produto e das prioridades, ciclos de iteração curtos orientados ao valor, gestão consciente dos compromissos entre rapidez e qualidade e responsabilidade ponta-a-ponta: do desenho ao *deploy* e à monitorização. Essas aprendizagens, técnicas e organizacionais, sustentam a evolução futura do sistema e da equipa.

Bibliografia

[1] OnlyHighIQ, *OnlyHighIQ*, https://onlyhighiq.com, 2025.

[2] R. G. Espinha, *Scrum – O guia definitivo da metodologia ágil*, Artia Blog, Available: https://artia.com/blog/scrum. Accessed: Aug. 21, 2025, abr. de 2024.

[3] S. Sachdeva, «Scrum Methodology,» *International Journal of Engineering and Computer Science*, vol. 5, n.º 6, pp. 16 792–16 799, jun. de 2016, issn: 2319-7242. doi: 10.18535/ijecs/v5i6.11. URL: https://doi.org/10.18535/ijecs/v5i6.11.

[4] Google Developers, *Client Libraries | YouTube Data API*, Available at https://developers.google.com/youtube/v3/libraries, (Último acesso em 07/09/2025), 2025.

[5] Kick, *Gaules | Kick*, https://kick.com/gaules, set. de 2025.

[6] Wikipedia, *Twitch (service)*, https://en.wikipedia.org/wiki/Twitch_(service), (Último acesso em 07/09/2025), 2025.

[7] T. Interactive, *Video Broadcast | Twitch Developers*, https://dev.twitch.tv/docs/video-broadcast/, set. de 2025.

[8] Q. Chang, Z. Yang e Y. Song, *A scalable mobile live video streaming system based on RTMP and HTTP transmissions*, https://doi.org/10.1007/978-3-642-21402-8_18, (Último acesso em 07/09/2025), mai. de 2011.

[9] A. Bentaleb, Z. Zhan, F. Tashtarian, M. Lim, S. Harous, C. Timmerer et al., *Low latency live streaming implementation in DASH and HLS*, https://doi.org/10.1145/3503161.3548544, out. de 2022.

[10] J. K. Nurminen, A. J. Meyn, E. Jalonen, Y. Raivio e R. G. Marrero, *P2P media streaming with HTML5 and WebRTC*, https://doi.org/10.1109/INFCOMW.2013.6970739, abr. de 2013.

[11] R. Malan e D. Bredemeyer, *Functional Requirements and Use Cases*, https://www.bredemeyer.com/pdf_files/functreq.pdf, out. de 2001.

[12] E. Rashid e N. E. Mastorakis, *A Novel Complexity Scoring Model for Non-Functional Requirements in Request for Proposal in Housing Industry Using FURPS Quality and Moscow Priority Model*, https://housingscience.org/volume-45-issue-4/a-novel-complexity-scoring-model-for-non-functional-requirements-in-request-for-proposal-in-housing-industry-using-furps-quality-and-moscow-priority-model/, dez. de 2024.

[13] S. Brown, *Visualising Software Architecture with the C4 Model — Agile on the Beach 2019*, https://www.youtube.com/watch?v=x2-rSnhpw0g, (Último acesso em 09/09/2025), 2019.

Apêndice A

Diagrama de casos de uso e Diagramas de sequencia

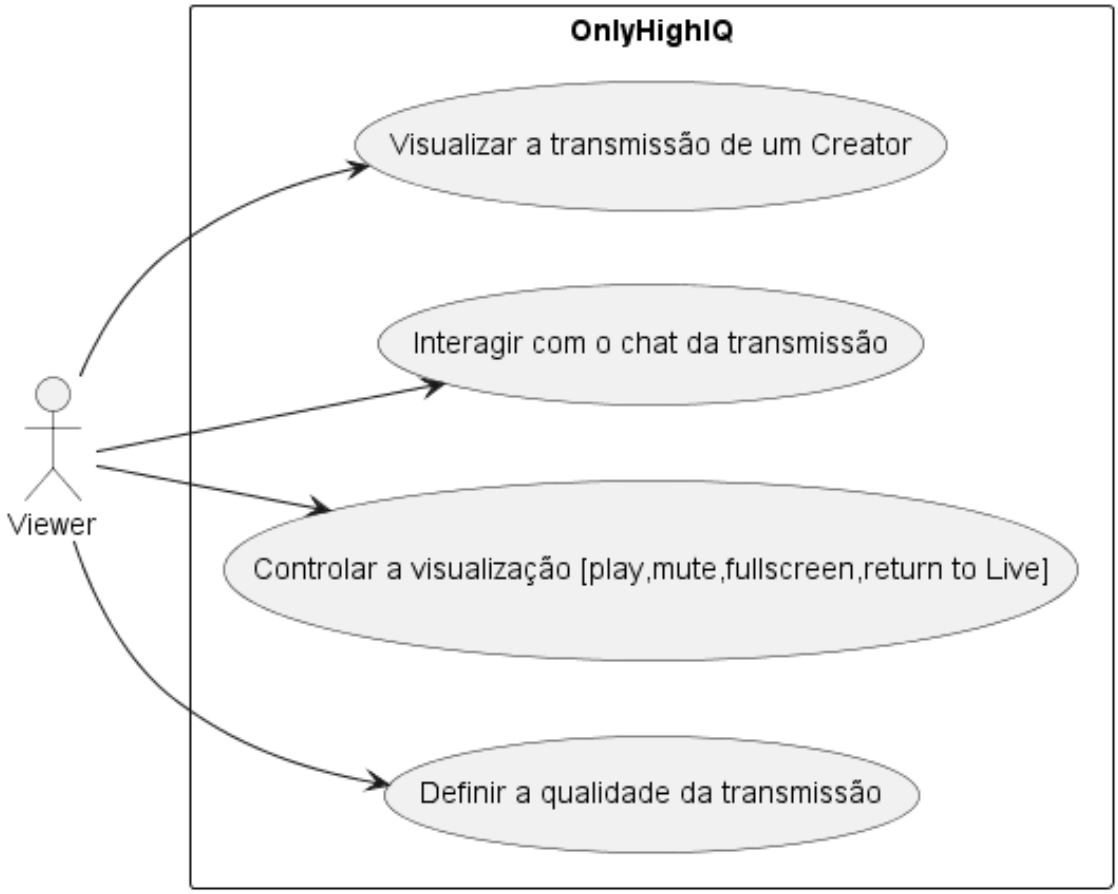

Figura A.1: Diagrama de casos de uso - Viewer

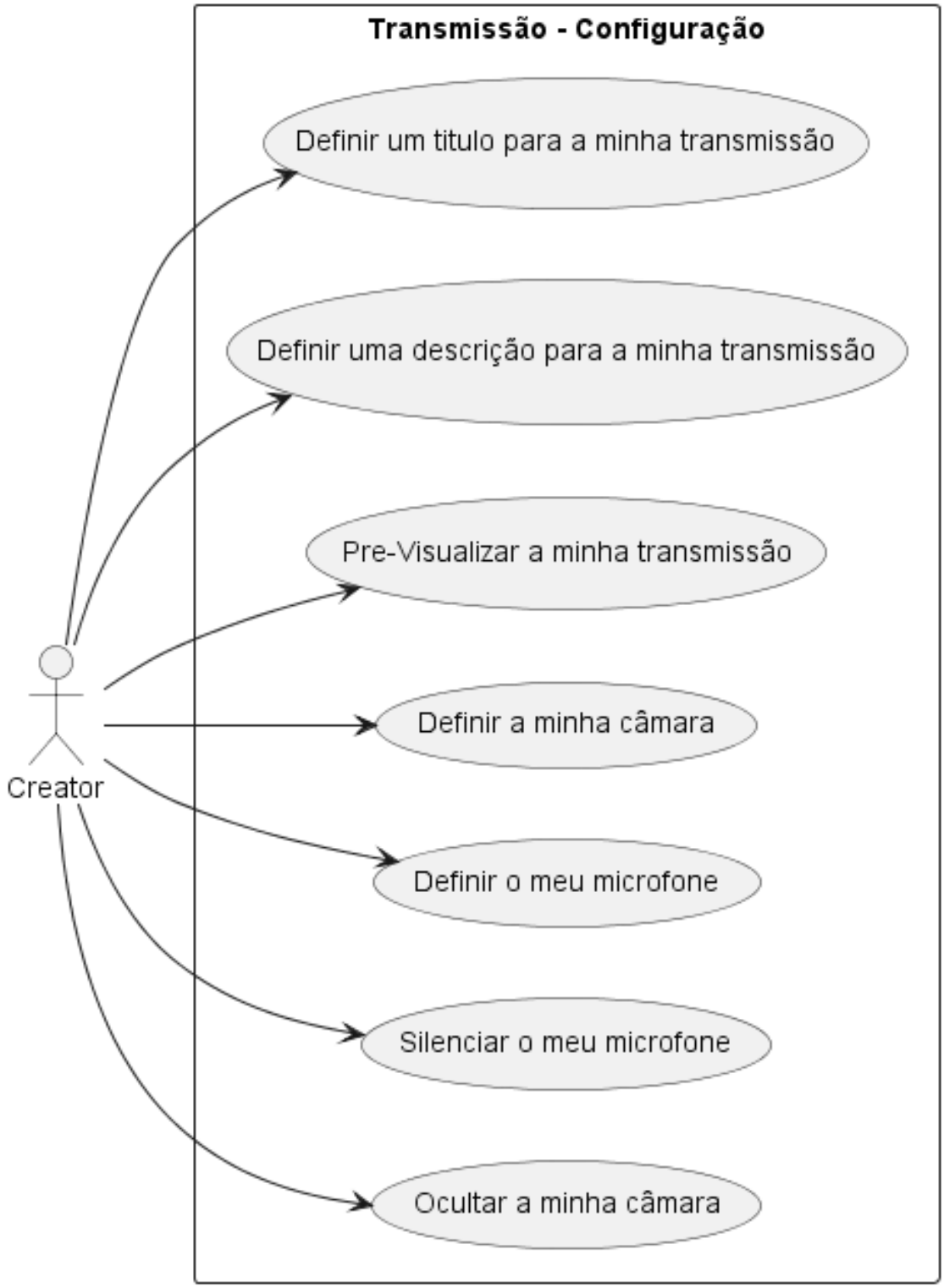

Figura A.2: Diagrama de casos de uso - Creator Configuração

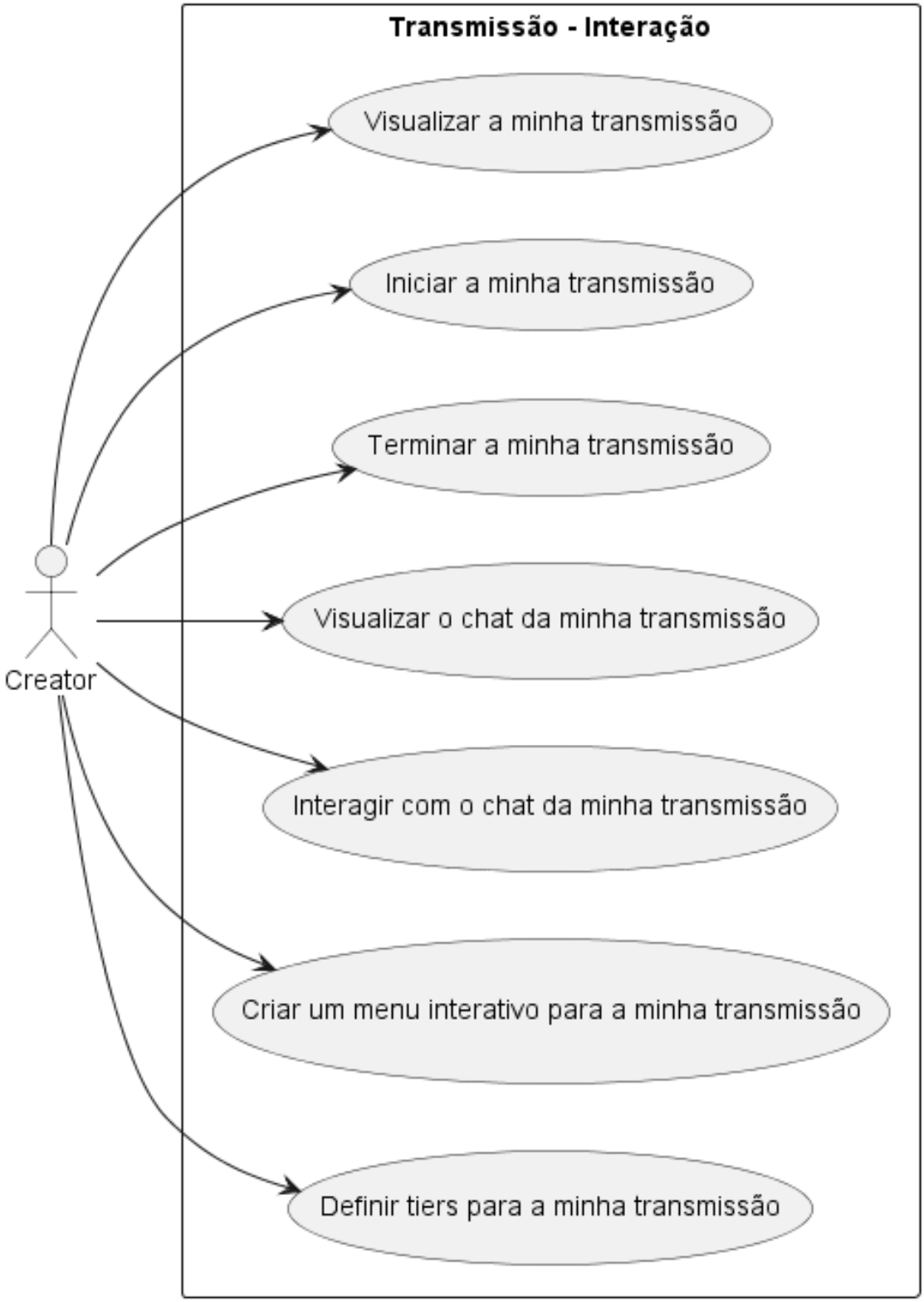

Figura A.3: Diagrama de casos de uso - Creator Interação

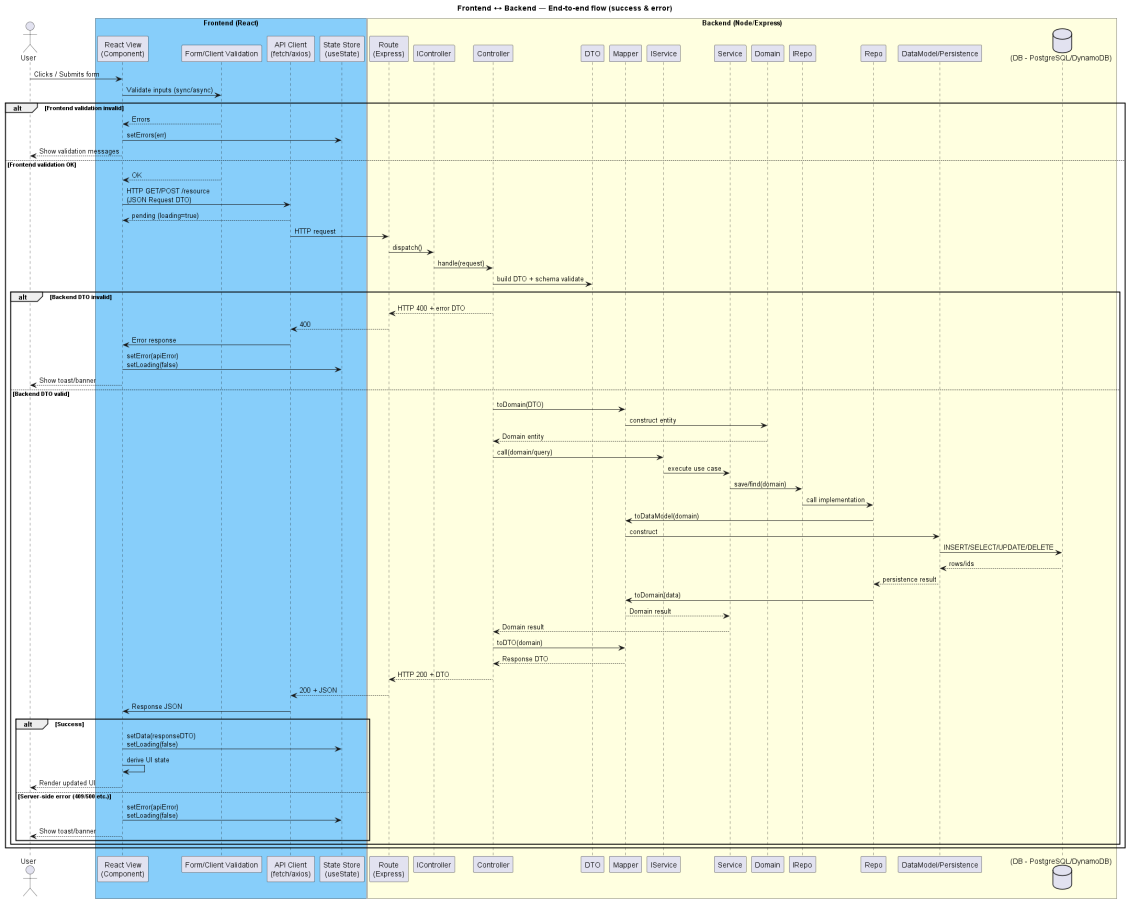

Figura A.4: Diagrama de Sequência completo

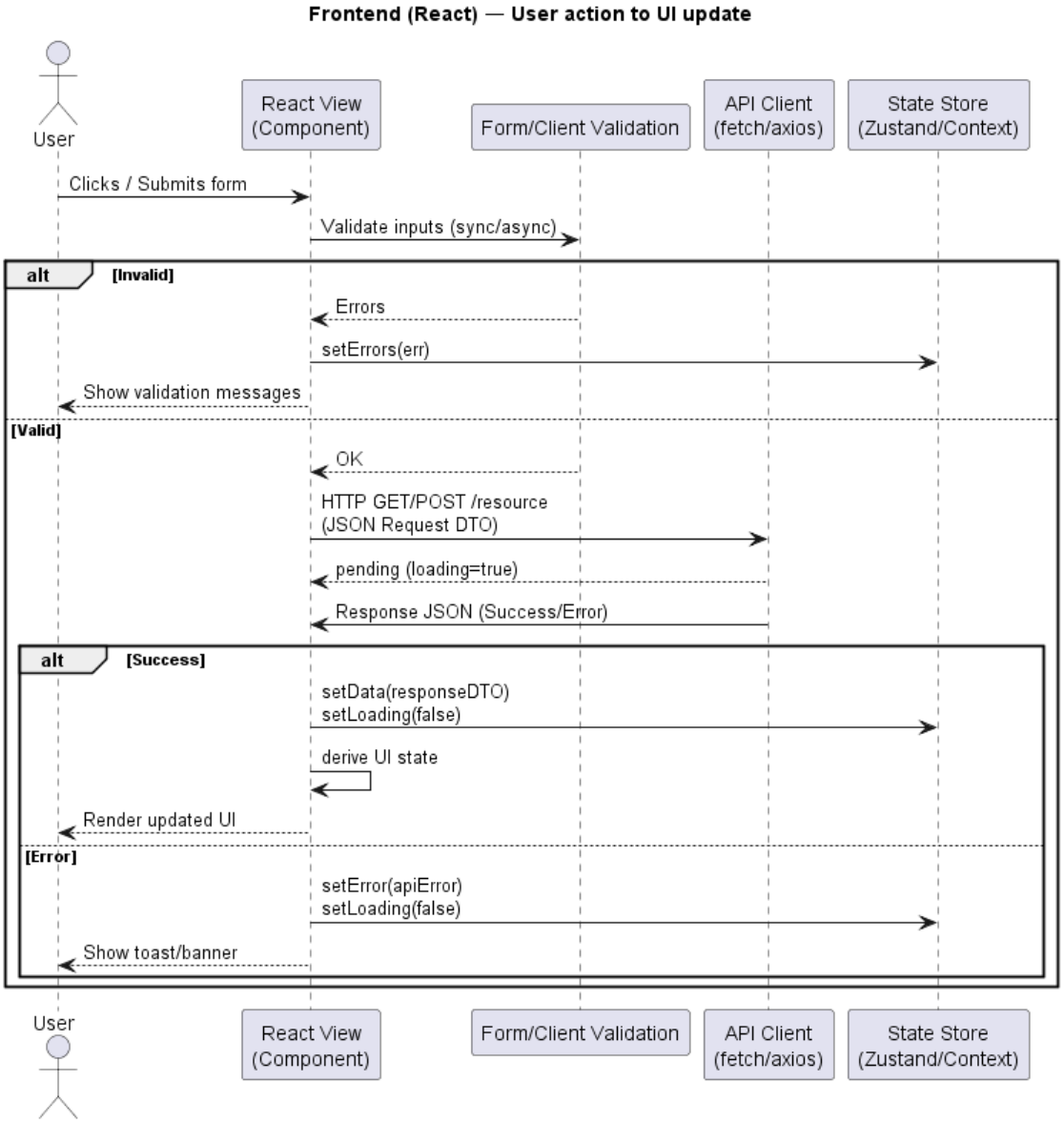

Figura A.5: Diagrama de Sequência - *Front-End*

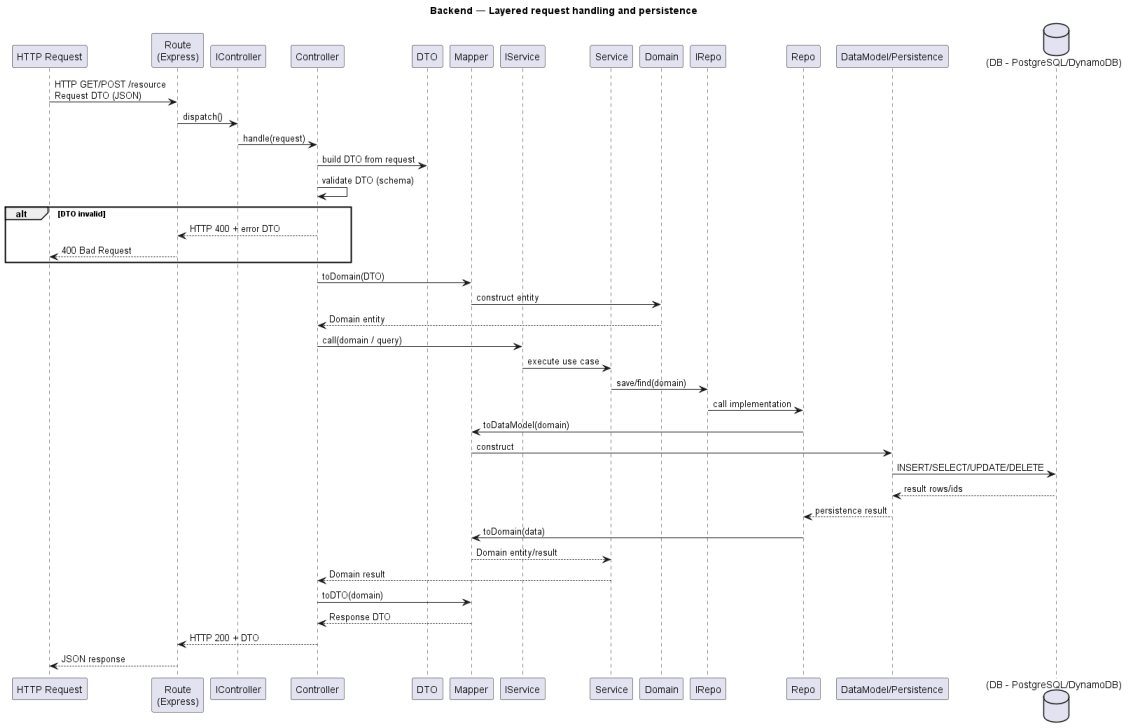

Figura A.6: Diagrama de Sequência - *Back-End*